

First Tree-like Quantum Data Structure: Quantum B+ Tree (Technical Report)

ABSTRACT

Quantum computing is a popular topic in computer science, recently attracting many studies in various areas such as machine learning, network and cryptography. However, the topic of quantum data structures seems long neglected. There is an open problem in the database area: Can we make an improvement on existing data structures by quantum techniques? Consider a dataset of key-record pairs. Given an interval as a query range, a B+ tree can report all the records with keys within this interval, which is called a *range query*. A classical B+ tree answers a range query in $O(\log N + k)$ time, where N is the total number of records and the output size k is the number of records in the interval. It is asymptotically optimal in a classical computer but not efficient enough in a quantum computer, because k (the output size) could be as large as $O(N)$.

In this paper, we propose the *quantum range query* problem. Unlike classical range queries, a *quantum range query* returns the range query results in *quantum bits*, which has broad potential applications due to the foreseeable future advance of quantum computers and quantum algorithms. To the best of our knowledge, we design the first tree-like quantum data structure called the *quantum B+ tree*. Based on this data structure, we propose a hybrid quantum-classical algorithm to do the range search. It answers a static quantum range query in $O(\log N)$ time, which is asymptotically optimal in quantum computers, where, here, “static” means the data is fixed and does not change over time. Since the execution time does not depend on the output size (i.e., k), it is significantly faster than the classical data structure. Moreover, we extend our quantum B+ tree to answer the dynamic quantum range queries efficiently in $O(\log^2 N)$ time. Our experimental results show that our proposed quantum data structures achieve up to 10,000x improvement in IO cost and 100x improvement in estimated execution time compared to their classical competitors.

ACM Reference Format:

. 2027. First Tree-like Quantum Data Structure: Quantum B+ Tree (Technical Report). In *Proceedings of SIGMOD 2027 (SIGMOD '27)*. ACM, New York, NY, USA, 17 pages. <https://doi.org/XXXXXXX>

1 INTRODUCTION

Consider a dataset where each item in this dataset is in the form of a key-record pair. The range query problem, which is to report all the items with keys within a given query range, has been a longstanding and widely-applied problem. For example, a teacher

may need to list all the students who obtained 40-60 marks in an exam, and a smartphone buyer may need to list all the smartphones that sit around \$200-\$400.

Apart from directly returning all the results to the user, range queries have also been applied as a subroutine of various problems. One typical example is the database retrieval with a range-based filtering condition. For instance, Alice wants to obtain the top ranking movies of 1990s in a movie dataset. To answer that, range queries are commonly performed first to extract all the feasible items (e.g., all movies with release year between 1990 and 1999) as candidates and then the desired items (e.g., the movies with the highest ranking score) are retrieved from these candidates.

To answer range queries, many data structures [8, 19, 49] have been proposed in classical computers to store the key-record pairs. One representative and widely-used data structure is the B+ tree [19]. It answers a range query in $O(\log N + k)$ time, where N is the total number of key-record pairs and the output size k is the number of records in the query range. Based on [65], the B+ tree is asymptotically optimal for range queries in classical computers. Note that for all range queries, the linear time of generating all the k results is inevitable, which is not efficient enough when range queries are used as subroutines of algorithms in quantum computers.

Recently, quantum algorithms have attracted a lot of attention. Many quantum algorithms [3, 28, 36, 40, 52, 57] have been proposed and are expected to show quadratic or even exponential speedup compared to classical algorithms, and thus many linear-complexity problems can be solved in sub-linear time in quantum computers.

For instance, one of the most significant advances in quantum algorithms for database search is Grover’s algorithm [28]. It searches for an item in a dataset containing N unordered items in $O(\sqrt{N})$ time. Follow-up studies solve database queries such as finding the maximum item (among N unordered items) also in $O(\sqrt{N})$ time [17, 22]. Note that traditional algorithms cannot achieve a better result than the $O(N)$ time of linear scanning for such database searches (since the items in the database are unordered).

Consider our database search example for finding the top-ranking movies of 1990s. Suppose that the number of candidates (i.e., moves of release year between 1990 and 1999) is k , which is resulted from a traditional range query. Although quantum algorithms could find the top ranking movies among the k candidates in $O(\sqrt{k})$ time efficiently, the time cost to list all the k candidates (which is inevitable in traditional range query) is $O(k)$. This limits the overall efficiency, since k could be very large.

Motivated by the critical limitations of the classical range query in quantum computers, we propose the problem called the *quantum range query*, which returns the answer in *quantum bits*, since we notice that quantum algorithms normally have the input form of quantum bits. Specifically, the quantum range query $QUERY(x, y)$ does not return a list of key-record pairs, but returns the quantum bits in a *superposition* of all the key-record pairs with keys within the query range $[x, y]$, where the superposition is the ability of a quantum

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '27, Jun 13–Jun 19, 2027, Hunting Beach, CA, USA

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN XXX...\$15.00

<https://doi.org/XXXXXXX>

system to be in multiple states simultaneously. For example, consider a movie dataset $\{(key_0, rec_0), \dots, (key_{N-1}, rec_{N-1})\}$, where N is the number of movies. Assume that we can construct a data structure in the quantum computer. If Alice wants to obtain all the movies of the 1990s, a quantum range query $QUERY(1990, 1999)$ will search on the data structure and return the quantum bits in a superposition of all the movies (key_i, rec_i) whose key_i is within the interval $[1990, 1999]$. Motivated by the different scenarios in real-world applications, we study the quantum range query problems in two different cases. The first case is that the dataset is *immutable*, so the problem is called the *static quantum range query*. The second case is that insertions and deletions are supported in the dataset, and the problem is called the *dynamic quantum range query*.

Note that our quantum range query returns results in quantum bits. This enables many potential applications, since the downstream quantum algorithms could leverage our results as input. The major applications are listed as follows. (1) Our quantum range query could be applied whenever a user wants to perform a database query among all *candidates* that satisfy a filtering condition on a numerical range (e.g., the aforementioned query of finding the movies having the highest rank with release date between 1990 and 1999). Other query types include simple search query (e.g., finding all movie directed by James Cameron in 90s) and aggregation query (e.g., counting or calculating the sum price of all orders with dates between 24 and 29 in Jan 2025). In the literature, many quantum algorithms have been proposed to support efficient database queries for maximum/minimum query [17, 22], search query [28], aggregation query [14, 43], etc. Our quantum range query retrieves candidates in qubits, enabling efficient execution of these algorithms to derive specific results. (2) Our quantum range query could also be applied in query processing and optimization of relational databases. Firstly, since most database queries can be expressed in SQL, the quantum range query can be integrated into quantum-based query processing pipelines. Secondly, for queries involving numerical range conditions, accurately estimating the number of rows under each condition is essential for selecting optimal query plans (e.g., determining the best join algorithm / join order). Our quantum range query could be applied to estimate the number of rows under the number range conditions efficiently. (3) Data binning [21] is a popular approach to improve the accuracy of various machine learning algorithms [11, 63], which applies range queries to group similar items within a query range (i.e., a data bin) together. Applying the quantum range queries, the quantum binning results in superposition can be *readily* used in quantum machine learning algorithms [3].

Besides the above applications, many quantum algorithms are designed to have input and output in the form of superpositions. For example, the HHL algorithm [31] returns the answer to a linear system of equations in a superposition, and it becomes a subroutine of the quantum SVM [51]. Thus, we believe that in the future, more quantum algorithms will benefit from the quantum range queries that return the query results in superposition.

The quantum range query problem has two distinctive characteristics. The first characteristic is that it allows the utilization of data structures for building an index to accelerate database queries, which has been a common approach in the database area. Existing

quantum algorithms [12, 22, 28, 29] focus on searching with unstructured data, which can handle small amounts of data but cannot solve the quantum range query problem efficiently in large databases. Among them, the best quantum range query algorithm adapted from [12] returns the result of k items in $O(\sqrt{Nk})$ time, which is inefficient when the dataset size N grows large. Note that although porting the entire database into the quantum computer to build the quantum data structures has an unignorable computational cost, it is considered as a pre-processing step (which corresponds to building a database index) and once this step is done, all the quantum range queries can be answered efficiently.

The second characteristic is the use of quantum computation to handle superpositions efficiently. As mentioned previously, the $O(\log N + k)$ time complexity for a range query in classical computer is already asymptotically optimal, where the $O(k)$ cost of listing out all the k results has no chance to be improved. In the quantum range query problem, however, since we aim to obtain the k results in the form of a superposition, it is possible to eliminate this $O(k)$ cost and thus achieve a better time complexity with the techniques in quantum computation.

Motivated by this, we propose the *quantum B+ tree*, which is the first tree-like quantum data structure to the best of our knowledge. We choose the B+ tree [19] as the first data structure to be studied on quantum computers. Since the B+ tree is one of the most fundamental and widely-used data structures, it is considered the most suitable one to open a new world of quantum data structures. We design our quantum B+ tree with two components, the classical component and the quantum component. The classical component follows the classical B+ tree, which allows us to leverage its effective balanced tree structure. The quantum component stores a concise “replication” of the hierarchical relationships in the B+ tree in the quantum memory, which could load the relationships in the form of superposition efficiently due to quantum parallelism.

Empowered by the two-component design of our quantum B+ tree, we propose a hybrid quantum-classical algorithm called the *Global-Classical Local-Quantum* (GCLQ) search to solve the quantum range query problem. It involves two main steps. The first step is called the *global classical search*, which finds at most two candidate nodes from the classical B+ tree. It is guaranteed that all the relevant results are covered in the candidate nodes and account for a significant amount under the candidate nodes (i.e., at least $\frac{1}{8B}$ of the items under candidate nodes are the relevant results). Meanwhile, the global classical search is very efficient owing to the effective structure of B+ tree. The second step is called the *local quantum search*, which returns a superposition of all the exact query results from the candidate nodes with efficient quantum parallelism techniques in the quantum memory. As a result, the time complexity of our proposed GCLQ search is $O(C_l \log N)$, where C_l denotes the cost of a load operation on the quantum memory, which is asymptotically optimal in quantum computers. Based on previous studies [32, 35, 37, 41, 42, 44, 45], C_l is commonly regarded as a constant since the quantum memory is believed to support very efficient load and store operations in future quantum computers, and thus the complexity can be simplified to be $O(\log N)$. This improves the optimal classical result by reducing the $O(k)$ cost and is much more efficient than the $O(\sqrt{Nk})$ time complexity of

the existing quantum algorithms without using any data structure. Furthermore, our quantum B+ tree has $O(N)$ space cost, as efficient as the classical B+ tree.

We also extend our quantum B+ tree to solve the dynamic quantum range query. Our two-component design allows us to flexibly extend our quantum B+ tree to the B+ tree variants. As such, we extend our quantum B+ tree to the *dynamic quantum B+ tree* by adapting the logarithmic method [10], which supports inserting a new item into the tree and deleting an existing item from the tree. All the insertions and deletions are also replicated to the quantum component of the tree, which is efficient due to the conciseness of the quantum component. Our dynamic quantum B+ tree efficiently handles each insertion and deletion in $O(\log N)$ time and performs the dynamic range query in $O(\log^2 N)$.

In summary, our contributions are shown as follows.

- We are the first to study the quantum range query problems.
- We are the first to propose a tree-like quantum data structure, which is the quantum B+ tree.
- We design a hybrid quantum-classical algorithm that can answer a quantum range query on a quantum B+ tree in $O(C_l \log N)$ ($\approx O(\log N)$) time, which does not depend on the output size and is asymptotically optimal in quantum computers.
- Our quantum B+ tree has efficient $O(N)$ space cost.
- We further extend the quantum B+ tree to the dynamic quantum B+ tree that supports insertions and deletions in $O(\log N)$ time and handles the quantum range query in $O(\log^2 N)$ time.
- We conducted experiments to confirm the significant speedup of the quantum range queries on real-world datasets. Our quantum data structure is up to 10,000x more efficient in IO cost than the classical data structure and 100x faster in the estimated execution time when the database size reaches over 100M.

The rest of the paper is organized as follows. In Section 2, we first introduce some basic knowledge about quantum algorithms. We formally define the quantum range query problems in Section 3. In Section 4, we show the design of the quantum B+ tree and introduce our algorithm to answer the quantum range query. In Section 5, we extend the quantum B+ tree to the dynamic version. Section 6 presents our experimental studies. In Sections 7 and 8, we introduce the related work and conclude our paper, respectively.

2 PRELIMINARIES

We first introduce the basic concepts of quantum bits and quantum circuit model in Section 2.1. Then, we introduce the concept of quantum random access memory in Section 2.2.

2.1 Quantum Circuit Model

A *bit* is a basic unit when we store data in the memory or on disks. In classical computers, a bit has two *states* (i.e., 0 and 1). A bit in quantum computers is known as a *quantum bit*, which is called a *qubit* in this paper for simplicity. Similar to a bit in classical computers, a qubit also has states. Following [8, 20], we introduce the “Dirac” notation (i.e., “ $|\cdot\rangle$ ”) to describe the states of a qubit. Specifically, we write an integer in $|\cdot\rangle$ (e.g., $|0\rangle$) to describe a *basis state*, and we write a symbol in $|\cdot\rangle$ (e.g., $|q\rangle$) to describe a *mixed state* (more formally known as a *superposition*). Note that the symbol in a superposition of a qubit is used to denote this qubit.

A qubit has two basis states, namely $|0\rangle$ and $|1\rangle$, which correspond to state 0 and state 1 of a classical bit, respectively. Different from a classical bit, a qubit q could have a state “between” $|0\rangle$ and $|1\rangle$, (described by a superposition $|q\rangle$). Specifically, superposition $|q\rangle$ is represented as a linear combination of the two basis states: $|q\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are two complex numbers called the *amplitudes* and we have $|\alpha|^2 + |\beta|^2 = 1$. In quantum computers, instead of directly obtaining the value of a bit, we *measure* a qubit q , which will “collapse” its superposition $|q\rangle$ and obtain the result state 0 (resp. 1) with probability $|\alpha|^2$ (resp. $|\beta|^2$). For instance, let q be a qubit with superposition $|q\rangle = (0.36+0.48i)|0\rangle + (0.48-0.64i)|1\rangle$. If we measure q , we can obtain 0 with probability $|0.36+0.48i|^2 = 0.36$ or 1 with probability $|0.48-0.64i|^2 = 0.64$.

Similar to a register in a classical computer, we define a *quantum register* to be a collection of qubits. Consider a quantum register with two qubits. The basis states of the two qubits will contain all the combinations of the basis states of single qubits, i.e., $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$. Similarly, the superposition of this two-qubit quantum register could be represented as a linear combination of these four basis states. If we measure one of the qubits, this measurement may change the state of the other qubit. Such phenomenon is widely known as *quantum entanglement* [55].

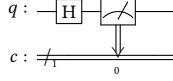
Consider the following example. For simple illustration, all the amplitudes in this example only have real parts (of complex numbers). Let ψ be a quantum register with two qubits q_0 and q_1 . Assume that ψ can be represented as $|\psi\rangle = |q_0q_1\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$. It indicates that ψ could collapse to two possible basis states $|00\rangle$ and $|11\rangle$ with equal probability (note that the amplitudes of the other two basis states $|01\rangle$ and $|10\rangle$ are 0). Thus, if q_0 is measured to be 0 (resp. 1), then the state of q_1 must also be 0 (resp. 1), which indicates that q_0 and q_1 are two *entangled* qubits.

We can extend this system to a quantum register with n qubits which have 2^n basis states and the corresponding 2^n amplitudes. Note that a quantum register, says ψ , can be represented as an ordered combination of all parts of ψ , says $\psi_1 \cdots \psi_m$ (where m denotes the number of parts). In this case, the quantum state of ψ can be written as $|\psi_1 \cdots \psi_m\rangle$ (or $|\psi_1\rangle \cdots |\psi_m\rangle$ interchangeably).

The states of qubits can be transformed by *quantum gates* in a *quantum circuit*, which work similarly as the gates and circuits in classical computers. This model is commonly known as the *quantum circuit model* [58]. We can *measure* a qubit in a quantum circuit. We introduce three common types of quantum gates that will be used in this paper, namely a *Hadamard gate*, an *X-gate* and a *controlled-X gate*. A *Hadamard gate* [30, 46] transforms $|0\rangle$ into $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$ and transforms $|1\rangle$ into $\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$. An *X-gate* [46] swaps the amplitudes of $|0\rangle$ and $|1\rangle$. A *controlled-X gate* has two parts, the *control* qubit, says q_1 , and the *target* qubit, says q_0 . A controlled-X gate applies an X-gate on the target qubit q_0 if the control qubit is $|1\rangle$, and otherwise, it keeps the target qubit unchanged.

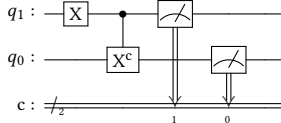
For example, assume an input $|q\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$. After being transformed by a Hadamard gate, we have $|q\rangle = \frac{1}{\sqrt{2}}(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle) + \frac{1}{\sqrt{2}}(\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle) = |0\rangle$. Clearly, if we now measure q , we will always obtain 0. We give more examples about the X-gate and the controlled-X gate shortly.

The following diagram illustrates an example of a quantum circuit, which consists of a qubit q , a timeline (denoted by a single-line wire) representing the quantum gates and operations applied on qubit q in order, a classical register c (denoted by a double-line wire), a Hadamard gate (denoted by a box containing “H”) and a measurement (denoted by a meter sign).



In this circuit, q is first transformed by a Hadamard gate and then measured by a measurement, where the measurement result is stored in the bit of position “0” (shown by the small number beneath the down arrow connecting the meter sign) of the register c whose number of bits is 1 (shown by the small number beside the “/” sign).

A quantum circuit could involve the combination of multiple input qubits and multiple gates. An example is shown as follows.



In this example, the input are two qubits q_0 and q_1 . We apply an X -gate on q_1 (denoted by the box containing “X”), and a *controlled-X* gate on q_0 and q_1 (denoted by the box containing “X^c”) where the control qubit is q_1 and the target qubit is q_0 . Consider the following three examples of input:

- (1) $|q_1 q_0\rangle = |00\rangle$. After the X -gate on q_1 , $|q_1 q_0\rangle = |10\rangle$. Since $|q_1\rangle = |1\rangle$, we apply an X -gate on q_0 , and thus the result is $|11\rangle$.
- (2) $|q_1 q_0\rangle = |11\rangle$. After the X -gate on q_1 , $|q_1 q_0\rangle = |01\rangle$. Since $|q_1\rangle = |0\rangle$, we do nothing, and thus the result is $|01\rangle$.
- (3) $|q_1 q_0\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$. After the X -gate on q_1 , $|q_1 q_0\rangle = \frac{1}{\sqrt{2}}|10\rangle + \frac{1}{\sqrt{2}}|01\rangle$. The result is $\frac{1}{\sqrt{2}}|11\rangle + \frac{1}{\sqrt{2}}|01\rangle$.

In classical computing, the circuit-level implementation details have been well developed and hidden in the hardware, and thus we only need to focus on the high-level logical operations like register allocation, arithmetic operations, control flow, memory management, etc. In contrast, quantum computing currently requires explicit design of quantum circuits for each operation or algorithm. Moreover, it remains open challenges for quantum computing to efficiently perform arbitrary combination of operations like control flow and memory management.

Therefore, in quantum computing, designing an algorithm typically involves designing a quantum circuit. Since the quantum circuit involves a timeline of gate and measurement operations, the time cost of a quantum circuit refers to the minimum number of steps to execute the entire circuit, while the space cost of a quantum circuit refers to the total number of qubits [59]. For instance, in the previous quantum circuit, the time cost is 3: first executing an X -gate on q_1 , then a controlled- X gate on q_0 and q_1 , and finally measuring both qubits (note that the controlled- X gate is dependent on the X -gate). The space cost is 1, as only one qubit is used.

2.2 Quantum Random Access Memory

In the study of traditional algorithms, we typically use the RAM model, where each primitive operation (e.g., store, load, arithmetic,

etc.) takes $O(1)$ time, and the time complexity of an algorithm is calculated to be the total number of primitive operations.

In this paper, we utilize *Quantum Random Access Memory* (QRAM) which is used to store the quantum data structure and give query results in superpositions. Like the classical RAM, we have QRAM in quantum computers to read and write quantum states. Following [36, 53], we assume the existence of a *classical-write quantum-read QRAM*. It can store a classical value into a given address. It can also accept a superposition of *multiple* addresses and return a superposition of the corresponding values due to quantum parallelism. Formally, we define QRAM as follows.

Definition 2.1 (Quantum Random Access Memory (QRAM)). A QRAM Q is an ideal model which can perform the following store and load operations.

- A store operation denoted by $Q[addr] = val$ stores value val into address $addr$, where $addr$ and val are two bit-strings.
- A load operation denoted by

$$Q \frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |addr_i\rangle |a\rangle = \frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |addr_i\rangle |a \oplus val_i\rangle \quad (1)$$

takes a superposition of a number k of addresses (i.e., $addr_i$ for $i \in [0, k-1]$) as input and loads a superposition of the values of these addresses into the destination quantum register of initial state a where $\oplus$ denotes the XOR operation.

In our definition of QRAM, the concept of store operation is similar to a classical RAM where we store val in a memory block at address $addr$. For simplicity, we also use $Q[addr]$ to denote the value stored at address $addr$. In the load operation, the input of multiple addresses and the output of multiple values are handled in superpositions, which is the major difference from a classical RAM. The result of a load operation is written to a quantum register using the XOR operation (i.e., $\oplus$). For instance, if we set the initial state of the destination quantum register to be $|0\rangle$, then the exact values will be loaded as follows.

$$Q \frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |addr_i\rangle |0\rangle = \frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |addr_i\rangle |Q[addr_i]\rangle \quad (2)$$

Efficient QRAM implementation remains an open challenge in quantum computing. Researchers have proposed various QRAM architectures with quantum circuits that support store and load operations, with the most prominent being the *Bucket Brigade QRAM* (BB-QRAM) [27]. In this architecture, three types of qubits are included, namely the data qubits for storage, the address qubits for accessing the addresses, and the routing qubits organized in a binary tree like structure to efficiently navigate addresses to the specific data qubits for superposition-based loading.

In particular, consider a BB-QRAM that can store 3-bit data of 2-bit addresses ($2^2 = 4$ memory blocks). To access data at address $|01\rangle$, the address qubits are set to $|01\rangle$, and then the first address qubit ($|0\rangle$) issues signal “left” at the root level of the routing qubits, while the second address qubits ($|1\rangle$) issues signal “right” at the next level, loading the data at $|01\rangle$. For multiple addresses, address qubits are set in superposition, and routing qubits split signals to enable parallel access to corresponding memory blocks.

While basic implementations of BB-QRAM achieve linear time cost for loading k memory blocks and $O(1)$ for a store operation [27], recent advances have optimized the loading time cost of k memory blocks approximately to logarithm of k [50]. For space cost, the BB-QRAM requires $n_d 2^{n_a}$ data qubits, 2^{n_a} routing qubits and n_a address qubits [27], where n_d denotes the number of bits per memory block, and n_a denotes the number of bits per address. In the previous example of BB-QRAM that can store 3-bit data of 2-bit addresses, the space cost (i.e., number of qubits required) is $3 \cdot 2^2 + 2^2 + 2 = 18$. Since n_d is typically a constant, it is easy to observe that BB-QRAM has $O(N)$ space cost for N memory blocks.

QRAM is an active topic in quantum computing under development, and researchers believe that future advances of quantum hardware and QRAM could support very efficient load and store operations. In particular, previous studies [32, 35, 37, 41, 42, 44, 45] indicate that each load operation, which does not have a theoretical complexity guarantee for now, will be very efficient in the future (e.g., with a nearly constant time cost). Thus, in this paper, we use C_l to denote the time cost of a load operation on QRAM.

3 PROBLEM DEFINITION

In this section, we formally define the quantum range query problems in the *static* and *dynamic* cases, respectively.

We first consider the *static* quantum range query problem (for simplicity, it is simply called the quantum range query problem). We are given an *immutable* dataset D of N items. Each item in D is represented as a key-record pair (key_i, rec_i) for $i \in [0, N-1]$. Note that the subscription i in (key_i, rec_i) represents the *index* of this pair, and the indices start from 0. Each key key_i is an integer, and each record rec_i is a bit-string. Note that in our problem, the keys are integers, but they can be easily extended to float numbers. A range query of lower bound x and upper bound y is denoted as $QUERY(x, y)$. In classical range query problem, $QUERY(x, y)$ returns a list of key-record pairs whose keys fall in range $[x, y]$. Let k be the number of items that $QUERY(x, y)$ returns, and let $l_0, l_1, \dots, l_{k-1}$ be the indices of the returned items. The returned list of $QUERY(x, y)$ is thus represented as $\{(key_{l_0}, rec_{l_0}), \dots, (key_{l_{k-1}}, rec_{l_{k-1}})\}$. In the quantum range query problem, we aim to return a superposition of all the key-record pairs in the list. We formally define the quantum range query problem as follows.

Definition 3.1 (Quantum Range Query). Given an immutable dataset $D = \{(key_0, rec_0), \dots, (key_{N-1}, rec_{N-1})\}$ and two integers x and y where $x \leq y$, a quantum range query $QUERY(x, y)$ is to return the following superposition

$$\frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |key_{l_i}\rangle |rec_{l_i}\rangle,$$

such that for each $i \in [0, k-1]$, $x \leq key_{l_i} \leq y$.

Note that in the quantum range query, we return a superposition which is the linear combination of all the desired key-record pairs. It can be observed that for each $i \in [0, k-1]$, the probability of obtaining $|key_{l_i}\rangle |rec_{l_i}\rangle$ is equal to $\frac{1}{k}$.

Next, we also define the *dynamic* quantum range query. In the dynamic case, instead of having an immutable dataset, we consider a *dynamic* dataset D that supports the *insertion* and *deletion* operations. Specifically, an insertion operation inserts a new item

(key, rec) into D and returns the new dataset $D \cup (key, rec)$. A deletion operation delete an existing item (key, rec) from D and returns the new dataset $D \setminus (key, rec)$. The dynamic quantum range query is formally defined as follows.

Definition 3.2 (Dynamic Quantum Range Query). Given a dataset $D = \{(key_0, rec_0), \dots, (key_{N-1}, rec_{N-1})\}$ that supports the insertion and deletion operations and two integers x and y where $x \leq y$, a dynamic quantum range query $QUERY(x, y)$ is to return the following superposition

$$\frac{1}{\sqrt{k}} \sum_{i=0}^{k-1} |key_{l_i}\rangle |rec_{l_i}\rangle,$$

such that for each $i \in [0, k-1]$, $x \leq key_{l_i} \leq y$.

Note that the only difference between the dynamic and the static quantum range query is whether the given dataset is dynamic.

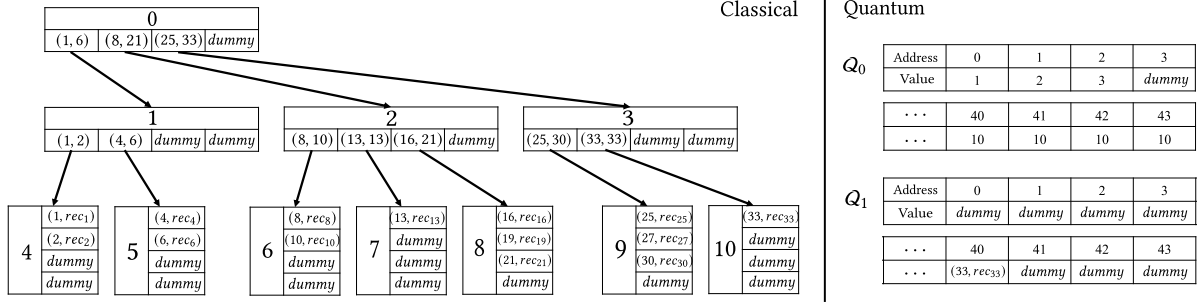
In our quantum range query problems, we aim to return a superposition of all the desired results, which is new in the research area to the best of our knowledge. Although it requires additional steps for quantum range queries to list all answers (e.g. by measuring the superposition result $O(k \log k)$ times [25]), we still consider it very meaningful, since we do not regard the result as a final output available to a user but as input to other efficient quantum algorithms, such as quantum database queries, SQL processing, query optimization and quantum machine (as mentioned in Section 1).

4 QUANTUM B+ TREE

In this section, we propose the quantum B+ tree to solve the quantum range query problems. To the best of our knowledge, no prior studies explored using a data structure in quantum computers. Compared with performing a quantum algorithm on *unstructured* dataset, using quantum data structures could answer our quantum range query problems more efficiently. We focus on the static quantum B+ tree in this section, and discuss the dynamic variant later in Section 5.

With the concept of QRAM (introduced in Section 2.2), we can form the following method of answering the quantum range query problems in two steps using the existing quantum algorithms on the *unstructured* dataset. In the first step of pre-processing (for loading the dataset), for each key-record pair (key_i, rec_i) , we store it into a QRAM Q using the store operation $Q[key_i] = rec_i$. Clearly, the total time complexity of pre-processing is $O(N)$, since we perform N store operations totally. In the second step of data retrieval, we first find all the addresses in Q that are within the query range, which costs $O(\sqrt{Nk})$ time using the state-of-the-art quantum algorithm [12, 15], and then use the load operation to find the resulting superposition of the corresponding values. Totally, the data retrieval step costs $O(\sqrt{Nk} + C_l)$ where C_l denotes the time cost of a load operation in QRAM. When N grows large, this is even worse than the classical range query algorithms (e.g., using the B+ tree) with time complexity logarithm to N .

To solve the quantum range query problems more efficiently with QRAM, we propose our quantum data structure called the quantum B+ tree. The major design of our quantum B+ tree is derived from the basic idea of the classical B+ tree. It is well known that the classical B+ tree utilizes a balanced tree structure, which performs a range query in the dataset D of size N efficiently with

Figure 1: An Example of a Quantum B+ Tree where $N = 14$ and $B = 4$

time complexity $O(\log N + k)$, where k is the return size. In classical computers, this time complexity is shown to be asymptotically optimal. For the quantum range queries in quantum computers, we are also interested in the best possible time complexity. To explore that, we introduce another problem called the *membership* problem, which, given a query key, decides whether this key exists in the dataset D . Existing studies have shown that the time complexity lower bound of solving the membership problem in quantum computers is $\Omega(\log N)$ [5, 56]. Based on that, we present the following lemma to show the time complexity lower bound of solving our quantum range queries. For the sake of space, we give a proof sketch for each lemma or theorem in this paper, and the full proof can be found in our technical report [7].

LEMMA 4.1. *The time complexity to answer a quantum range query is $\Omega(C_l \log N)$, where C_l is the cost of a load operation on QRAM.*

PROOF SKETCH. Consider a quantum range query $QUERY(x, x)$. It answers whether key x exists, which is a membership problem of time complexity $\Omega(C_l \log N)$ by [56]. Therefore, the time complexity of answering a quantum range query is also $\Omega(C_l \log N)$. $\square$

Based on [32, 35, 37, 41, 42, 44, 45], C_l is commonly regarded as a constant, and thus $\Omega(C_l \log N)$ could be simplified as $\Omega(\log N)$.

According to the above lemma, we cannot achieve a better time complexity than $\Omega(\log N)$ for the quantum range query. The goal of our static quantum B+ tree is thus to obtain this asymptotically optimal time complexity $\Omega(\log N)$. Since the $O(k)$ time cost of listing all the k returned results cannot be reduced in classical range queries, we consider eliminating this cost in our quantum B+ tree, which is feasible in quantum computers that handle the superpositions due to quantum parallelism.

Based on the above analysis, in the following, we first introduce the details of how we design our quantum B+ tree in Section 4.1, and then, we propose our quantum range search algorithm called the *global-classical local-quantum search* in Section 4.2.

4.1 Design of Quantum B+ Tree

The design of our quantum B+ tree includes the following two core ideas. Firstly, a quantum B+ tree consists of the classical component and the quantum component. The classical component stored in a classical computer is the same as the classical B+ tree, and the quantum component stores a concise replication of the hierarchical relationships and node data in the classical component. This is to take advantage of the balanced tree structure of the classical B+ tree,

which has a tree *height* of $O(\log N)$ and thus retains the $O(\log N)$ search efficiency. Secondly, the quantum component utilizes QRAM to process the tree structure and return the range search results in superpositions efficiently by quantum parallelism.

We first introduce some basic concepts of the classical B+ tree, which is the prototype of our quantum B+ tree. We consider a *weight-balanced B+ tree* [6], a tree data structure that consists of nodes in two types, the internal nodes and the leaves. Each node has a unique ID in the form of an integer to represent this node. A user parameter B called the branching factor is set to be an integer that is a power of 2 and is at least 4. A leaf contains B key-record pairs sorted by the keys in ascending order. An internal node has B children, each of which is a leaf or an internal node. Each node is also associated with a routing key, says (L, U) , representing a range with lower bound L and upper bound U , and all the key-record pairs under the internal node with routing key (L, U) have keys within this range. The children under an internal node have non-overlapping routing keys and are sorted by the lower bound of their routing keys. A node or a key-record pair under a leaf could be empty, which is represented as a special mark called *dummy*. We assume that *dummy* appears only in the last positions under a node.

The left part in Figure 1 illustrates an example of the classical component of a classical B+ tree with $N = 14$ key-record pairs and the branching factor $B = 4$. It has four internal nodes of ID from 0 to 3 and seven leaves of ID from 4 to 10. For instance, the node of ID 1 (simply called node 1) has routing key $(1, 6)$, indicating that all the data items with keys in range $[1, 6]$ are under node 1. Also, node 1 has four children, two of which are non-dummy leaves and the other two are dummy nodes. For the leaf shown as node 4 with routing key $(1, 2)$, two data items (with keys equal to 1 and 2) and two dummy items are assigned.

We define the *level* of a node to be its distance to the root, where the distance between two nodes is defined to be the minimum number of edges connecting these two nodes. In a B+ tree, all the leaves have the same level. We also define the *height* of a node to be its distance to a leaf, and the height of the B+ tree is defined to be the height of the root. We define the *weight* of a node to be the number of all non-dummy key-record pairs under this node. A non-root node of height h is said to be *balanced* if its weight is between $\frac{1}{4}B^{h+1}$ and B^{h+1} .

In our running example shown in the left part of Figure 1, there are 4 non-dummy key-record pairs under node 1 of height h equal

to 1, and thus the weight of node 1 is equal to 4. This indicates that node 1 is balanced (since 4 is no less than $\frac{1}{4}B^{h+1} = 4$).

A B+ tree is said to be a weight-balanced B+ tree if all its non-root nodes are balanced and its root has at least two non-dummy children. It is easy to verify that the height of a weight-balanced B+ tree with N key-records pairs is $O(\log N)$. In Figure 1, the B+ tree shown in the left part is an example of a weight-balanced B+ tree.

After introducing the structure of a classical B+ tree, we now discuss how we represent the hierarchical relationships and node data of a B+ tree in QRAM. Since each node contains B elements (including dummy), each of which has a child node with a routine key or a key-record pair, we could fit a classical B+ tree into a QRAM storing $M * B$ key-value pairs, where M denotes the total number of nodes in the classical B+ tree, a key represents the ID of a tree node and a value stores the related information of this node. This can be done by a traversal of all the tree nodes in any order (in this paper, we choose the breadth-first order for simplicity).

We use two QRAMs, namely the hierarchy QRAM Q_0 and the data QRAM Q_1 , to store the hierarchical relationships (i.e., the mapping from each node to its children) and the data (i.e., the routine key of a node or the key-record pair of a leaf), respectively. Specifically, we perform $M * B$ store operations on Q_0 where M denotes the total number of nodes in the classical B+ tree as follows. Let $C_{i,j}$ (for $i \in [0, M-1]$ and $j \in [0, B-1]$) denote the node ID of the j -th child of node i in the B+ tree. Specially, if node i is a leaf with no children, we set $C_{i,j} = i$. Also, if the j -th child of node i is *dummy*, we assign *dummy* to $C_{i,j}$. Then, for each node $i \in [0, M-1]$ and each $j \in [0, B-1]$, we perform a store operation to store value $C_{i,j}$ at address $i * B + j$, i.e., $Q_0[i * B + j] = C_{i,j}$.

In the right part of Figure 1, we show an example of the quantum B+ tree. In the hierarchy QRAM Q_0 , totally 44 ($= 11 * 4$) addresses are “occupied”. Among them, for instance, node 0 involves storing the IDs of its three non-dummy children (i.e., 1–3) and one *dummy* into addresses 0–3, respectively, and node 10 involves storing value 10 for all its corresponding addresses 40–43 since it is a leaf.

The data QRAM Q_1 stores the routine key of each node or the key-record pair of each leaf. Similar to Q_0 , we also perform $M * B$ store operations on Q_1 . Consider node i in the B+ tree. If i is an internal node, then for its j -th child, we assign the routine key (i.e., a 2-tuple of lower bound and upper bound of this routine key) to a variable, says $P_{i,j}$, which is *dummy* if the j -th child is dummy. If i is a leaf, we set $P_{i,j}$ to be the j -th key-record pair of node i or *dummy* if the j -th key-record pair is dummy. As such, for each node $i \in [0, M-1]$ and each $j \in [0, B-1]$, we perform a store operation to store pair $P_{i,j}$ at address $i * B + j$, i.e., $Q_1[i * B + j] = P_{i,j}$.

Continuing our running example in Figure 1, in the data QRAM Q_1 , we also store values into 44 addresses similarly, where for node 0, since it is an internal node, all values stored are *dummy* (at addresses 0–3), and for node 10 with two key-record pairs and two dummies, the values at address 40–43 are stored correspondingly.

In the following lemma, we show the memory consumption of our quantum B+ tree in QRAM.

LEMMA 4.2. *The total number of memory blocks of the weight-balanced quantum B+ tree in QRAM is $O(N)$.*

PROOF SKETCH. Since each leaf node contains at least $\frac{B}{4}$ (and at most B) key-record pairs, the number of leaf nodes is thus $O(\frac{N}{B})$. According to [38], the number of non-leaf node of a weight-balanced B+ tree is also $O(\frac{N}{B})$. Since each node consumes B memory blocks in QRAM, the total number of memory blocks is $O(N)$. $\square$

Based on this lemma, it is clear that loading the quantum B+ tree into QRAM takes $O(N)$ time, since each load operation costs $O(1)$.

Note that both QRAMs have data across the same number (i.e., $M * B$) of addresses, and at the same address, the value stored are related to the same node or the same key-record pair in a leaf. Thus, it is easy to combine the two QRAMs into one QRAM by using a larger quantum register storing the values of both QRAMs at the same address for implementation. In this paper, for clear structuring and illustration, we use two QRAMs to separate them.

For both Q_0 and Q_1 , we could easily retrieve multiple values for the relationships in the tree with one load operation on QRAM. Specifically, in the node QRAM Q_0 , we perform the following load operation for node i ($i \in [0, M-1]$)

$$\begin{aligned} Q_0 \frac{1}{\sqrt{B}} \sum_{j=0}^{B-1} |i * B + j\rangle |0\rangle &= \frac{1}{\sqrt{B}} \sum_{j=0}^{B-1} |i * B + j\rangle |C_{i,j}\rangle \\ &= \frac{1}{\sqrt{B}} \sum_{j=0}^{f_i-1} |i * B + j\rangle |C_{i,j}\rangle + \frac{1}{\sqrt{B}} \sum_{j=f_i}^{B-1} |i * B + j\rangle |dummy\rangle \end{aligned} \quad (3)$$

where f_i denotes the number of non-dummy children among all the children of node i . This operation loads the IDs of all the children of node i into a superposition in C_i time. Similarly, on Q_1 , we can load the key-record pairs under a leaf (with node ID i) with the following load operation

$$\begin{aligned} Q_1 \frac{1}{\sqrt{B}} \sum_{j=0}^{B-1} |i * B + j\rangle |0\rangle &= \frac{1}{\sqrt{B}} \sum_{j=0}^{B-1} |i * B + j\rangle |P_{i,j}\rangle \\ &= \frac{1}{\sqrt{B}} \sum_{j=0}^{f_i-1} |i * B + j\rangle |P_{i,j}\rangle + \frac{1}{\sqrt{B}} \sum_{j=f_i}^{B-1} |i * B + j\rangle |dummy\rangle. \end{aligned} \quad (4)$$

4.2 Global-Local Quantum Range Search

In this section, we propose an algorithm called the *global-local quantum range search* to solve the quantum range query with our quantum B+ tree.

Recall that our quantum range query aims to return a superposition of all the key-record pairs such that each key in the result is within a query range $[x, y]$. To better present our algorithm, we first propose a technique called *post-selection* which can answer the quantum range query with our dataset stored in a QRAM even in an *unstructured* way (i.e., without using the quantum B+ tree).

The major idea of post-selection is based on the efficient processing of superpositions in QRAM. Specifically, we first apply the store operations to store all the key-record pairs in the dataset D in a QRAM, says Q' , which is a *pre-processing* step. That is, for each $i \in [0, N-1]$, we do a store operation $Q'[i] = (key_i, rec_i)$. We can thus obtain a superposition of all the key-record pairs as $\frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |key_i\rangle |rec_i\rangle$ by a load operation on Q' in C_l time. After obtaining this superposition, we design a quantum circuit as follows to *post-select* the superposition of the desired items within $[x, y]$.

Let in denote the set of indices of all the key-record pairs in the dataset such that the keys are within $[x, y]$, and let out denote the set of all the remaining indices. First, we place an additional qubit with initial state $|0\rangle$ in the end for the purpose of controlling and measuring. Thus, we obtain $\frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |key_i\rangle |rec_i\rangle |0\rangle$. Next, we perform a quantum circuit which transforms the above superposition based on a controlled-X gate controlled by whether key_i is within $[x, y]$. Specifically, if $x \leq key_i \leq y$, the last qubit will be transformed to $|1\rangle$, and otherwise, no transformation is performed. After the transformation, we obtain $\frac{1}{\sqrt{N}} \sum_{i \in in} |key_i\rangle |rec_i\rangle |1\rangle + \frac{1}{\sqrt{N}} \sum_{i \in out} |key_i\rangle |rec_i\rangle |0\rangle$. Note that k (resp. $N - k$) denotes the size of in (resp. out). The above state can be rewritten as $\frac{\sqrt{k}}{\sqrt{N}} \cdot \frac{1}{\sqrt{k}} \sum_{i \in in} |key_i\rangle |rec_i\rangle |1\rangle + \frac{\sqrt{N-k}}{\sqrt{N}} \cdot \frac{1}{\sqrt{N-k}} \sum_{i \in out} |key_i\rangle |rec_i\rangle |0\rangle$, where the part $\frac{1}{\sqrt{k}} \sum_{i \in in} |key_i\rangle |rec_i\rangle$ is exactly the quantum range query result we desire. Finally, we measure the last qubit, which will obtain the desired superposition of all the key-record pairs in set in (i.e., $\frac{1}{\sqrt{k}} \sum_{i \in in} |key_i\rangle |rec_i\rangle$) with probability equal to $\frac{k}{N}$ (i.e., the last qubit is measured to be 1). Since our desired result could not be deterministically obtained, we repeat the final measurement step until the last qubit is measured to be 1. It is easy to verify that N/k repeats are needed in expectation.

Overall, to answer the quantum range query, the online processing of post-selection has expected time complexity at least $O(N/k)$, since the final step is repeated N/k times in expectation. In range queries, the number of results k is normally much smaller than the dataset size N , and thus the $O(N/k)$ time could be costly. Therefore, the post-selection technique as a standalone approach is inefficient, since the dataset is organized in an unstructured manner in QRAM.

However, if $k \approx N$, the $O(N/k)$ cost could be small (e.g., close to an $O(1)$ cost) for a post-selection. Motivated by this, we propose our efficient quantum range query algorithm based on our quantum B+ tree, which leverages post-selection as a sub-step.

Our quantum range query algorithm is called the Global-Classical Local-Quantum search (GCLQ search). It has two major steps. The first step is the *global classical search*, which aims to find the candidate B+ tree nodes *precisely* from the classical B+ tree such that they contain all the query results but the number of irrelevant results is as small as possible. The second step is the *local quantum search*, which searches from the QRAMs in the quantum B+ tree and returns the result in superposition efficiently with a post-selection step. In the following subsections, we introduce the two steps.

4.2.1 Global Classical Search. We first introduce the global classical search step. In this step, we traverse the classical B+ nodes from the root to obtain the “precise” candidate nodes that are without too many irrelevant results. Although the classical B+ is already capable of returning all the exact results, we do not expect returning many detailed-level candidate nodes in this step (which could cause the undesired cost of listing all the candidate nodes). Instead, we postpone the processing of detailed-level nodes for fast local search in the quantum component, and we only return at most two candidate nodes which are as “precise” as possible.

Specifically, given a B+ tree node with routing key (L, U) and the query range $[x, y]$, we classify this node into three categories,

namely an *outside* node if range $[L, U]$ and range $[x, y]$ have no overlap, a *partial* node if $[L, U]$ is partially inside $[x, y]$, and an *inside* node if $[L, U]$ is completely inside $[x, y]$. A partial node i is said to be *precise* if i is a leaf or there exists an inside node among the children of node i . It can be verified that if a partial node is precise and the B+ tree is weight-balanced, then a sufficiently significant portion of key-record pairs under this node are included in the query range $[x, y]$ (which is formalized in a lemma shortly).

Now, we present the detail of our global classical search, which returns the *precise* partial nodes with levels as small as possible (to avoid processing detailed-level nodes). For simplicity, we assume that the root node is a partial node (otherwise the results will involve no item or all items). We create an empty list $\mathcal{L}$ of tree nodes and add the root to the list initially. Then, we loop the following steps until $\mathcal{L}$ is empty. (1) We check whether there exists a precise node in $\mathcal{L}$. (2) If the answer of Step (1) is yes, we immediately add all nodes in $\mathcal{L}$ into the returned candidate set and terminate. Otherwise, we replace each node in $\mathcal{L}$ with all its non-outside children (since outside nodes cannot contain the desired results).

For example, consider a query $QUERY(5, 11)$ given to the quantum B+ tree in Figure 1. We first check the root (i.e., node 0) with three children. We find that two of them (i.e., node 1 and 2) are partial nodes and one of them (i.e., node 3) is an outside node. Thus, the root is not precise (i.e., there does not exist a precise node in $\mathcal{L}$), and we replace the root with node 1 and node 2 in $\mathcal{L}$. Then, we check whether node 1 or node 2 is precise. We find that node 2 is precise since it contains an inside child (i.e., node 6). Therefore, we obtain the returned candidate set containing node 1 and node 2.

The following lemma shows the effectiveness of the global classical search.

LEMMA 4.3. *Given a query range $[x, y]$ and a B+ tree that is weight-balanced, the returned candidate set of the global classical search contains at most two nodes. Moreover, let $\mathcal{R}$ be the set of all key-record pairs under the above returned candidate nodes, and let $\mathcal{R}^*$ be the set of all key-record pairs such that the keys are within the query range $[x, y]$. Then, $\mathcal{R}^* \subset \mathcal{R}$ and $|\mathcal{R}^*| \geq \frac{1}{8B} |\mathcal{R}|$.*

PROOF SKETCH. Since the routing keys of all nodes in the same level are disjoint, we cannot have more than two partial nodes in the same level. It is also easy to verify that the returned nodes are from the same level. Thus, the candidate set contains at most two nodes. Since we only filter out the outside node in this algorithm, we have $\mathcal{R}^* \subset \mathcal{R}$. Finally, since the precise partial node either is a leaf (which contains at least $1/B$ items in the query range), or contains an inside child (which contains at least $\frac{1}{4B}$ items in the query range due to the balanced nodes of the B+ tree), and it can be verified that at least one returned node is precise, we have $|\mathcal{R}^*| \geq \frac{1}{8B} |\mathcal{R}|$. $\square$

4.2.2 Local Quantum Search. Now, we introduce the local quantum search. Since the local quantum search starts from at most two candidate nodes, consider answering $QUERY(x, y)$ starting from a node u and another node v as the candidate nodes in level j and the height of the tree is h . Step 1 is to initialize the first n_i quantum qubits to be $\frac{1}{\sqrt{2}}(|u\rangle + |v\rangle)$, where n_i denotes the number of bits to store each node index i . Then, in Step 2, we add $n_b = \log_2 B$ auxiliary qubits $|0\rangle$ to the last and also apply a Hadamard gate on

each auxiliary qubit. We obtain

$$\frac{1}{\sqrt{2}}(|u\rangle + |v\rangle)\mathcal{H}|0\rangle\mathcal{H}|0\rangle\cdots\mathcal{H}|0\rangle = \frac{1}{\sqrt{2B}}(|u\rangle + |v\rangle) \sum_{i=0}^{B-1} |i\rangle.$$

This step is to enumerate all the edges from the candidate nodes. Then, in Step 3, we add n_i qubits to the end and apply Q_0 to obtain all the children of the candidate nodes, so we obtain

$$\frac{1}{\sqrt{2B}}|u\rangle \sum_{i=0}^{B-1} |i\rangle |c_i\rangle + \frac{1}{\sqrt{2B}}|v\rangle \sum_{i=0}^{B-1} |i\rangle |c_{i+B}\rangle,$$

where $c_0, \dots, c_{B-1}$ are the B children of u and $c_B, \dots, c_{2B-1}$ are the B children of v . If we only look at the last n qubits, we obtain $\frac{1}{\sqrt{2B}} \sum_{i=0}^{2B-1} |c_i\rangle$, which is the $2B$ children of u and v . We repeat Step 2 and Step 3 for $h-j$ times so that we obtain all the $2^{B^{h-j}}$ leaves below u and v . Then, we do the same thing as Step 2 to enumerate all the $2^{B^{h-j+1}}$ key-record pairs in the $2^{B^{h-j}}$ leaves. In the last step, we apply Q_1 to obtain all the key-record pairs below u and v and then do a post-selection search. Denote the $2^{B^{h-j+1}}$ key-record pairs as $(key_0, rec_0), \dots, (key_{2^{B^{h-j+1}}-1}, rec_{2^{B^{h-j+1}}-1})$. Then the quantum state becomes $\frac{1}{\sqrt{2^{B^{h-j+1}}}} \sum_{i=0}^{2^{B^{h-j+1}}-1} |key_i\rangle |rec_i\rangle$. We also use $|in\rangle$ to denote the k key-record pairs in the query range and use $|out\rangle$ to denote the other dummy key-record pairs and non-dummy key-record pairs which are not in the query range. We obtain $\frac{\sqrt{k}}{\sqrt{2^{B^{h-j+1}}}} |in\rangle + \frac{\sqrt{2^{B^{h-j+1}}-k}}{\sqrt{2^{B^{h-j+1}}}} |out\rangle$. If we do a post-selection, we can obtain $|in\rangle$ with probability $\frac{k}{2^{B^{h-j+1}}}$.

For example, consider a query $QUERY(5, 11)$ on the quantum B+ tree in Figure 1. As mentioned in Section 4.2.1, the candidate nodes are node 1 and node 2. First, we initialize $|\psi\rangle = \frac{1}{\sqrt{2}}(|1\rangle + |2\rangle)$. After applying Hadamard gates and Q_0 , we obtain $|\psi\rangle = \frac{1}{\sqrt{8}}(|4\rangle + |5\rangle + |6\rangle + |7\rangle + |8\rangle) + \frac{\sqrt{3}}{\sqrt{8}} |dummy\rangle$, which consists of all the children of node 1 and node 2. Then, after applying Hadamard gates and Q_1 , we obtain all the key-record pairs below node 1 and node 2, which is $|\psi\rangle = \frac{1}{\sqrt{32}}(|1\rangle |rec_1\rangle + |2\rangle |rec_2\rangle + |4\rangle |rec_4\rangle + |6\rangle |rec_6\rangle + |8\rangle |rec_8\rangle + |10\rangle |rec_{10}\rangle + |13\rangle |rec_{13}\rangle + |16\rangle |rec_{16}\rangle + |19\rangle |rec_{19}\rangle + |21\rangle |rec_{21}\rangle) + \frac{\sqrt{22}}{\sqrt{32}} |dummy\rangle$. Finally, by a post-selection, we can obtain $|\psi\rangle = \frac{1}{\sqrt{3}}(|6\rangle |rec_6\rangle + |8\rangle |rec_8\rangle + |10\rangle |rec_{10}\rangle)$ with probability $\frac{3}{32}$.

Finally, we show the time complexity of our GCLQ algorithm in the following theorem.

THEOREM 4.4. *On average, the quantum range query algorithm returns the answer in $O(C_l \log N)$ time.*

PROOF SKETCH. In the global classical search, we return at most two candidates at each tree level and the height of the tree is $O(\log N)$. Thus, the global classical search involves $O(\log N)$ load operations of QRAM (each has C_l cost) and costs $O(C_l \log N)$ time. In the local quantum search, we need to repeat all the steps in at most $8B$ iterations based on Lemma 4.3, which is a constant. In each iteration, we repeat Step 2 and Step 3 at most $O(\log N)$ times, and each repetition involves executing a quantum circuit for post-selection. Therefore, the total time complexity is $O(C_l \log N)$. $\square$

Similarly, this time complexity result can be simplified as $O(\log N)$ since C_l can be regarded as a constant and thus skipped according to [32, 35, 37, 41, 42, 44, 45]. By Theorem 4.4 and Lemma 4.1, this

algorithm is asymptotically optimal in a quantum computer. It is worth mentioning that the worst-case time cost of the global classical search part is also $O(\log N)$ due to the balanced structure of the classical component of the B+ tree.

Furthermore, by the following theorem, we also show that the space cost of our proposed quantum B+ tree is efficient.

THEOREM 4.5. *The space cost of the quantum B+ tree is $O(N)$.*

PROOF SKETCH. Based on Lemma 4.2, the number of memory blocks for the quantum B+ tree in QRAM is $O(N)$, indicating $O(N)$ space cost. Based on [38], the space cost of a classical B+ tree is $O(N)$. In the local quantum search, we only introduce $2n_i + n_b$ extra qubits, which is a constant. Finally, the space cost is $O(N)$. $\square$

5 DYNAMIC VARIANT

In this section, we introduce how to make the static quantum B+ tree dynamic, which solves the dynamic range query. Due to space limit, we only describe its general idea and leave more details of the dynamic variant in our technical report [7].

Since the design of our quantum B+ tree is to retain a classical B+ tree structure and to maintain a concise replication of hierarchical relationships in QRAMs for efficient retrieval of data records into superposition, we can apply this idea to the dynamic cases. In this sense, we adapt the idea of the logarithmic method [10] to allow insertions and deletions while maintaining tree balance and scalability through hierarchical merging.

Specifically, we build a list of $L = \lfloor \log_B N \rfloor + 1$ forests, each of which contains at most $B-1$ static quantum B+ trees. To insert a new key-record pair (key, rec) , we first insert it into a sorted list. When the length of this sorted list reaches B , we add it into F_0 . Then, whenever a forest F_i has B quantum B+ trees, which indicates that we have B quantum B+ trees of height i , we merge the B quantum B+ trees of height i into a quantum B+ trees of height $i+1$, and move it into F_{i+1} . Furthermore, we replicate the same insertion operations in quantum components of the quantum B+ trees. To delete a key-record pair, we first locate the quantum B+ tree in the forests that contains the key-record pair, by maintaining a data structure (e.g., a hash) to store their mappings. Then, we delete the key-record pair in the corresponding quantum B+ tree while keeping the corresponding tree balanced with suitable rebalancing operations.

To answer a quantum range query on the dynamic quantum B+ tree, we first perform a global classical search on each quantum B+ tree of each forest (each search returning at most two candidate nodes). Then, we merge all the candidate nodes into a combined quantum state and perform a quantum local search to obtain all the results in superposition.

Due to the efficient structure of the dynamic quantum B+ tree, each insertion and each deletion costs $O(\log N)$ and the total space cost of $O(N \log N)$. Since we search through $O(\log N)$ forests, a query on the dynamic quantum B+ tree costs $O(\log^2 N)$ time.

6 EXPERIMENT

In this section, we show our experimental results to verify the superiority of our proposed quantum B+ tree on quantum range queries. As mentioned in Section 2, a QRAM with N memory blocks needs $O(N)$ qubits. Even the most advanced quantum computers (e.g.,

the IBM’s 1121-qubit quantum processor [16]) only have around 1K qubits, which are not fully available for commercial or academic usage. Hence, we used simulation for experiments instead of real quantum computers, to verify the potentials of our quantum data structures with more than 100M data records (as introduced shortly).

In this paper, we mainly have two measurements. (a) *IO Cost*: All quantum-based and classical-based algorithms could be measured by the *IO cost*. Following a well-established common practice in the database community of measuring the performance of an operator (e.g., the cost of building an index [47, 64, 66], the cost of performing a query over an index [9, 34, 66] and the estimated cost of an evaluation plan for query optimization [18]) using the number of page accesses (simply the *IO cost*), we chose to evaluate the *number of memory accesses* as the *IO cost* to make the comparisons (since we also follow the basic common assumption of these prior works that all data has been loaded in the memory). In the quantum data structures, a QRAM read or write operation is counted as 1 IO, because researchers believe that with the future advances of quantum computers, the store and load operations of QRAM can be very efficient [32, 35, 37, 41, 42, 44, 45]. In the classical data structures, a page access is counted as 1 IO as well.

(b) *Execution Time*: Though the IO cost is an important measurement in the database community, it is also interesting to measure the execution time of all quantum-based and classical-based algorithms even if the *practical* implementation quantum computer involving a lot of qubits is still under development. Specifically, although there exist some quantum simulators such as Qiskit [62] and Cirq [26], they are not capable of simulating an efficient QRAM for executing our quantum range queries, because these simulators can hold a very limited number of qubits, which support only queries with only several data items, each with at most 2-3 bits. Thus, we chose to use C++ to perform the quantum simulations. We estimated the execution time of each of our quantum-based algorithms as follows.

- (1) We construct all the quantum circuits involved in the execution of quantum range queries.
- (2) In each of the above quantum circuits, we obtain a list of all quantum gates of various types that need to be run sequentially.
- (3) We measure the running time of each type of a quantum gate in a real circuit simulated by Qiskit [62].
- (4) We estimate the running time of each quantum circuit by summing up the measured running time of each gate.

Finally, the execution time is estimated to be the combination of the original running time (of our C++ quantum simulation) and the running time of all quantum circuits. Note that for each QRAM store or load operation, we also followed the above steps to construct its quantum circuit, where we used a state-of-the-art efficient quantum circuit implementation of the Bucket Brigade QRAM model [50]. In the classical data structures, we measured the real execution time for each query. Additionally, we measured the IO cost and execution time for index building of non-dynamic data structures and insertion/deletion of the dynamic data structures. The PC we used is equipped with an Apple M3 Pro CPU and 18GB memory.

We used three real datasets: (1) *Brightkite* from SNAP [39], containing 4M check-in records, each with a timestamp (integer) and location (2-tuple of integers). For scalability tests, we created a 128M

synthetic dataset with the same format and distribution. Dataset *Brightkite* is used for (quantum) range queries to retrieve (a superposition of) data items within a timestamp range. (2) *IMDb* [1], containing 1.4M movie records with a release year (integer) and rating (real number), was used in queries of finding specific movies (e.g., with highest rating or with specific title) within a release year range, which is aimed at verifying the superiority of using our quantum range query result in superposition as the input to the quantum algorithm. (3) *TPC-H* [2], a database containing multiple table related to a set of purchase orders, with 1.2M lineitems in its largest table, was used in query optimization to test how quantum range query results in superposition improve query performance by efficiently guiding query plan selection.

We compared our GCLQ search algorithm on our proposed quantum B+ tree and its dynamic variant (simply denoted as **quantum B+ tree**) with the classical B+ tree search algorithm and its dynamic version [10] (which are denoted as **classical B+ tree**). For queries related to applying the quantum range query result to downstream algorithms (e.g., finding the maximum data item), we compare several methods for ablation studies (described in detail later).

The factors we studied are the dataset size (i.e., N), the branching factor (i.e., B) and the number of data items within the query range (i.e., k). In dynamic quantum queries, we also studied the deletion-insertion ratio, which is defined to be the chance to delete an existing record (instead of doing an insertion), while we inserted all the data items in the dataset. For dataset *Brightkite*, we varied N from 4K to 128M, by randomly choosing a subset from the original (resp. synthetic) data items when $N \leq 4M$ (resp. $N > 4M$). B is varied from 4 to 64 for each data structure, and k is varied from 1% of N to 10% of N . The deletion-insertion ratio is varied from 1% to 5%. By default, $N = 2M$, $B = 8$, $k = 5\%$ of N and the deletion-insertion ratio is 1% (following [4]). For datasets *IMDb* and *TPC-H*, we set N and k with slight difference as described later. For each type of range query, we randomly generated 10,000 range queries and report the average measurement.

In the following, we present our experimental results.

Effect of N . We first studied the effect of the dataset N . As shown in Figures 2(a), while our proposed **quantum B+ tree** fails to achieve better performance for smaller N (e.g., $N < 100K$), it scales significantly better for both IO cost and execution time as N grows from 100K to 128M on dataset *Brightkite*, which verifies its efficient $O(\log N)$ cost of performing the GCLQ search. As N grows, the number of items within the query range (i.e., k) increases linearly, leading to linear growth in IO cost and execution time for the classical B+ tree, consistent with its $O(\log N + k)$ complexity. In contrast, our quantum B+ tree shows only slight growth in IO cost and execution time due to quantum parallelism, as the GCLQ algorithm’s classical global search costs $O(\log N)$, while quantum operations (e.g., loading the superposition from QRAM and performing the quantum local search) are nearly independent of N and k . Consequently, for $N = 128M$, our quantum B+ tree achieves around 10,000x less IO cost and 100x faster execution time compared to the classical B+ tree. Moreover, as shown in Figure 2(b), our **quantum B+ tree** incurs higher IO cost and time to build the index due to the additional construction of the quantum component in QRAM.

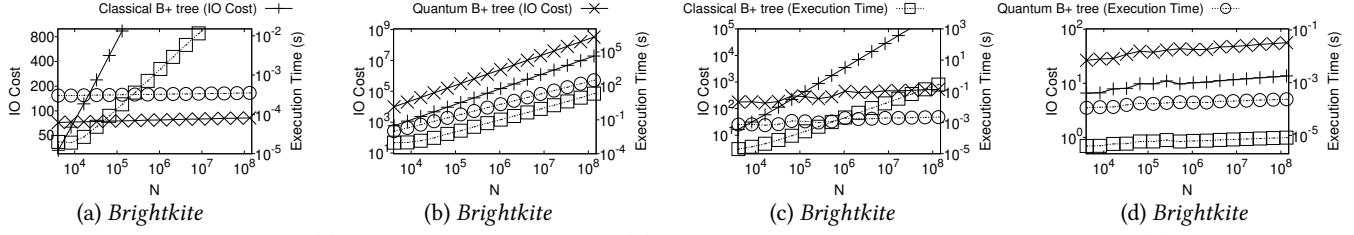


Figure 2: The Effect of N for (a) Quantum Range Queries, (b) Index Building of the Static Data Structures, (c) Dynamic Range Queries and (d) Insertions and Deletions into the Dynamic Data Structures on Dataset *Brightkite*

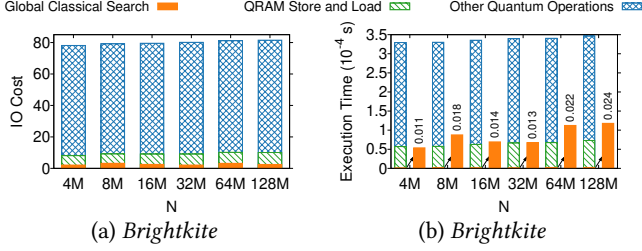


Figure 3: The Breakdown (a) IO Cost and (b) Execution time of Quantum B+ Tree for Quantum Range Queries on Dataset *Brightkite*

However, it maintains an efficient, linear construction cost similar to the **classical B+ tree**.

For dynamic cases, our dynamic **quantum B+ tree** achieves similar efficiency with much better scalability with N than the dynamic **classical B+ tree** (Figure 2(c)). Figure 2(d) shows it needs 3x higher IO cost and time for insertion or deletion due to operations on the quantum component, but these remain efficient with $O(N)$ growth. Insertion and deletion costs are similar and unaffected by varying the deletion-insertion ratio (see [7] for details).

Breakdown Cost. Taking a closer look at the cost of our **quantum B+ tree** for quantum range queries, the global classical search step shows the smallest cost with $O(\log N)$ growth (Figure 3), as at most two candidate nodes are kept per tree level. Among the quantum steps, loading candidate records in superposition from QRAM incurs less cost than other quantum operations, as the current QRAM implementation efficiently loads up to $8Bk$ records (based on Lemma 4.3) once in logarithm time (which is believed to be even faster in the future). The quantum local search requires a post-selection step executed $8B$ times, but since B is a small constant, quantum costs scale well with N , while the **classical B+ tree** suffers significant cost growth with k (and N correspondingly).

Effect of k and B . We studied the effect of k and B on quantum range queries. As shown in Figure 4(a), the IO cost and execution time of the **classical B+ tree** increase linearly due to its $O(\log N + k)$ query complexity, while our **quantum B+ tree** remains nearly independent of k , which is consistent with the analysis in the varying- N results. For increasing B , the **classical B+ tree** sees reduced IO cost (Figure 4(b)) due to fewer internal nodes being accessed. However, the overall execution time basically remains unchanged, since the time for processing each node is increased. Our **quantum B+ tree** needs more IO cost and time as B increases, because we need to $8B$ iterations of post-selection to obtain the final superposition of all desired records. When $B = 128$, our **quantum B+ tree** even has higher cost than the **classical B+ tree**. However, large B is known

to cause bulk memory consumption in classical B+ trees [19]. Thus, we set $B = 8$ in other experiments for fair comparisons with reasonable memory consumption and execution time. Similar results were observed in the dynamic case. Overall, our proposed quantum data structures favor a smaller branching factor.

Next, we show the results of applying our quantum range queries to downstream database queries and query optimization.

Search / Maximum Query. We conducted search and maximum queries on dataset *IMDb*, which retrieve movies with a specific title, or with the maximum rating, among the k candidates within a random release date range. We evaluated three algorithms: (1) using the **quantum B+ tree** to find all candidates in superposition before performing a quantum search / maximum algorithm (i.e., the Grover’s algorithm or its variants [17, 22, 28]) to find the answer, (2) using the **classical B+ tree** to find candidates in classical bits, followed by the quantum algorithm, and (3) using **classical B+ tree** with a linear search to find the answer.

Figure 4(c) and (d) show the results of the maximum query, with similar results for the search query (see [7] for details). The quantum maximum algorithm efficiently finds the highest-rating candidate in $O(\sqrt{k})$ time [17, 22], significantly reducing query time compared to a linear scan. However, using the **classical B+ tree** to retrieve k candidates requires k store operations to convert them into qubits before applying the quantum maximum algorithm, resulting in high IO costs for both baselines. In comparison, using quantum maximum algorithm with our **quantum B+ tree** eliminates any need for iterating through the k candidates, since **quantum B+ tree** returns the k candidates efficiently, which is ready to be used in the quantum maximum algorithm. It achieves an overall $O(\log N + \sqrt{k})$ time cost and the shortest execution time.

Moreover, our **quantum B+ tree** with quantum maximum algorithm scales well with N and k , which complies with our previous findings. Particularly, when N scales to 1M and k scales to 10% of N , the IO cost and time improvements are around 360x and 4.7x, respectively, compared with the best performing baseline.

Query Optimization. We explored using our **quantum B+ tree** for query optimization on dataset *TPC-H*, where we chose two typical scenarios, join order selection and join algorithm selection. For join order selection, we ran a query where Table *Lineitem* (1.2M records) is joined with both Table *Orders* (300K records) and *Partsupp* (160K records), applying where-clauses on *totalprice* in *Orders* and *availqty* in *Partsupp* within random ranges. The join began with the table containing fewer feasible records to minimize intermediate results for better performance. Similar experiments for join algorithm selection showed similar results, detailed in [7] due to space constraints.

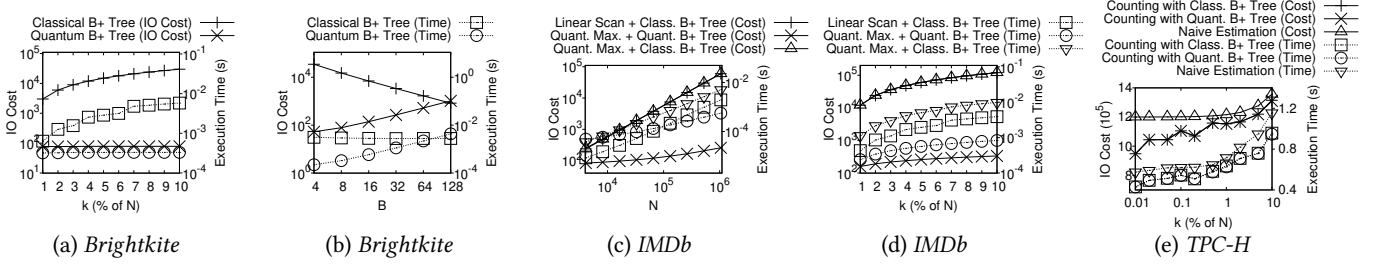


Figure 4: The Effect of (a) k and (b) B on Quantum Range Queries on Dataset *Brightkite*, the Effect of (c) N and (d) k of Maximum Query on Dataset *IMDb*, and (e) Query Optimization Results for Join Order Selection on Dataset *TPC-H*

For query optimization, it is essential to estimate the number of feasible records in tables efficiently before executing a query. We use our **quantum B+ tree** with a quantum counting algorithm [13] applied to the GCQL results to achieve this. To show its superiority, we compared it with two baselines: (1) using the **classical B+ tree** for counting, and (2) estimating feasible records based on the proportion of the query range in the where-clause to the entire attribute range. To simulate real-world scenarios, we vary the number of feasible records k from 0.01% to 10% of N (with N remained unchanged to be the total number of records).

As shown in Figure 4(e), proportional estimation incurs significantly higher IO costs and longer execution times due to the non-uniform data distribution and thus the frequent incorrect estimations, resulting in inefficient join orders. Our **quantum B+ tree** still obtains actual performance improvement over the **classical B+ tree** in counting feasible records, leveraging the quantum efficiency of the GCQL and quantum counting algorithms, which avoid traversing all k feasible records. For instance, with $k = 5\%N$, the **classical B+ tree** requires around 1,700 IOs for counting, while the **quantum B+ tree** needs fewer than 20. We also achieve around 32x faster time for the counting under this setting.

Summary. In summary, our **quantum B+ tree** with our proposed GCLQ search algorithm achieves up to 10,000x IO cost improvement and 100x execution time improvement over the **classical B+ tree** for answering the quantum range queries. The similar superiority is observed on the dynamic quantum range queries compared with the classical data structures. Our **quantum B+ tree** scales well with the growth of N and k . For applying our quantum range queries with various downstream tasks such as database queries and query optimizations, our **quantum B+ tree** together with the downstream quantum algorithms obviously outperforms the baselines.

7 RELATED WORK

The classical range query problem has been long studied with various data structures proposed to solve this problem, with little room for further improvement. For the static range query and the dynamic range query, the B-tree [8] and the B+ tree [19] are widely-used. As outlined in Section 1, the B+ tree solves the range query in $O(\log N + k)$ to find the k results out of N key-record pairs, which is asymptotically optimal in classical computers [65], but is not efficient enough for quantum algorithms due to linear cost with k .

The quantum database searching problem is well-known in quantum algorithms, with Grover’s algorithm [28] addressing the search for a marked record in an unstructured list of N records arranged randomly. Classically, $N/2$ queries to a function f (where $f(i) = 1$

if the i -th record is marked, and $f(i) = 0$ otherwise) are needed on average, while Grover’s algorithm reduces this to $O(\sqrt{N})$ queries by leveraging quantum parallelism. The algorithm iteratively amplifies the amplitude of the marked state, requiring $O(\sqrt{N})$ iterations to achieve that the probability of success is close to 1. An improved version [12] extends this to k -marked records, finding one in $O(\sqrt{N/k})$ time. When f marks records within a query range, it returns one of the k key-record pairs in $O(\sqrt{N/k})$ time and answers range queries in $O(\sqrt{Nk})$ time. However, as query time grows with $\sqrt{N}$, this method becomes inefficient for large N , especially compared to classical data structures with $O(\log N + k)$ complexity, as it does not exploit pre-built database indices.

In the database area, many studies explore using quantum computers to improve traditional database queries. For example, [23, 54, 60, 61] discuss quantum query optimization, leveraging quantum algorithms like quantum annealing [24] to optimize classical database queries. However, in the context of range queries, the idea of these studies will be to optimize range queries on classical computers, where the query returns a list of records. In this paper, we focus on using a hybrid algorithm to optimize range queries on a quantum computer, where the query returns quantum bits in a superposition of records. To our knowledge, this is the first study in the database area to explore quantum algorithms in this direction.

In conclusion, the existing classical data structures and the existing quantum database searching algorithms cannot solve the range query problem in quantum computers perfectly.

8 CONCLUSION

In this paper, we study the quantum range query problem. We propose the quantum B+ tree, the first tree-like quantum data structure, and the efficient global-classical local-quantum search algorithm based on the quantum B+ tree. Our proposed data structure and algorithm can answer a quantum range query in $O(\log N)$ time, which is asymptotically optimal in quantum computers and is exponentially faster than classical B+ trees. Furthermore, we extend it to a dynamic quantum B+ tree. The dynamic quantum B+ tree can support insertions and deletions in $O(\log N)$ time and answer a dynamic quantum range query in $O(\log^2 N)$ time. In the experiments, we did simulations to verify the superiority of our proposed quantum data structures compared with the classical data structures. We expect that the quantum data structures will show significant advantages in the real world.

The future work includes exploring even more efficient quantum algorithms for the dynamic and studying more advanced database queries such as the top- k queries.

REFERENCES

- [1] 2025. IMDb Non-Commercial Datasets. <https://developer.imdb.com/non-commercial-datasets/>.
- [2] 2025. TPC-Homepage. <https://www.tpc.org/>.
- [3] Soumik Adhikary, Siddharth Dangwal, and Debanjan Bhowmik. 2020. Supervised learning with a quantum classifier using multi-level systems. *Quantum Information Processing* 19, 3 (2020), 1–12.
- [4] Sattam Alsubaiee, Alexander Behm, Vinayak Borkar, Zachary Heilbron, Young-Seok Kim, Michael J Carey, Markus Dreseler, and Chen Li. 2014. Storage management in AsterixDB. *Proceedings of the VLDB Endowment* 7, 10 (2014), 841–852.
- [5] Andris Ambainis. 1999. A better lower bound for quantum algorithms searching an ordered list. In *40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039)*. IEEE, 352–357.
- [6] Lars Arge and Jeffrey Scott Vitter. 1996. Optimal dynamic interval management in external memory. In *Proceedings of 37th Conference on Foundations of Computer Science*. IEEE, 560–569.
- [7] Anonymous Author(s). 2023. *First Tree-like Quantum Data Structure: Quantum B+ Tree*. Technical Report. <https://github.com/anonym45263/EBDE402C7C7404A8>
- [8] Rudolf Bayer and Edward McCreight. 2002. Organization and maintenance of large ordered indexes. In *Software pioneers*. Springer, 245–262.
- [9] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R⁺-tree: An efficient and robust access method for points and rectangles. In *Proceedings of the 1990 ACM SIGMOD international conference on Management of data*. 322–331.
- [10] Jon Louis Bentley and James B Saxe. 1980. Decomposable searching problems I. Static-to-dynamic transformation. *Journal of Algorithms* 1, 4 (1980), 301–358.
- [11] Axel Berg, Magnus Oskarsson, and Mark O'Connor. 2021. Deep ordinal regression with label diversity. In *2020 25th International Conference on Pattern Recognition (ICPR)*. IEEE, 2740–2747.
- [12] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. Tight bounds on quantum searching. *Fortschritte der Physik: Progress of Physics* 46, 4-5 (1998), 493–505.
- [13] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. 2000. Quantum amplitude amplification and estimation. *arXiv preprint quant-ph/0005055* (2000).
- [14] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. 2002. Quantum amplitude amplification and estimation. *Contemp. Math.* 305 (2002), 53–74.
- [15] Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. Quantum counting. In *International Colloquium on Automata, Languages, and Programming*. Springer, 820–831.
- [16] Davide Castelvecchi. 2023. IBM releases first-ever 1,000-qubit quantum chip. *Nature* 624, 7991 (2023), 238–238.
- [17] Yanhu Chen, Shijie Wei, Xiong Gao, Cen Wang, Jian Wu, and Hongxiang Guo. 2019. An optimized quantum maximum or minimum searching algorithm and its circuits. *arXiv preprint arXiv:1908.07943* (2019).
- [18] Zhiyuan Chen, Johannes Gehrke, and Flip Korn. 2001. Query optimization in compressed database systems. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*. 271–282.
- [19] Douglas Comer. 1979. Ubiquitous B-tree. *ACM Computing Surveys (CSUR)* 11, 2 (1979), 121–137.
- [20] Paul Adrien Maurice Dirac. 1939. A new notation for quantum mechanics. In *Mathematical Proceedings of the Cambridge Philosophical Society*, Vol. 35. Cambridge University Press, 416–418.
- [21] James Dougherty, Ron Kohavi, and Mehran Sahami. 1995. Supervised and unsupervised discretization of continuous features. In *Machine learning proceedings 1995*. Elsevier, 194–202.
- [22] Christoph Durr and Peter Hoyer. 1996. A quantum algorithm for finding the minimum. *arXiv preprint quant-ph/9607014* (1996).
- [23] Tobias Fankhauser, Marc E Soler, Rudolf M Füchslin, and Kurt Stockinger. 2021. Multiple query optimization using a hybrid approach of classical and quantum computing. *arXiv preprint arXiv:2107.10508* (2021).
- [24] Aleta Berk Finnilla, MA Gomez, C Sebenik, Catherine Stenson, and Jimmie D Doll. 1994. Quantum annealing: A new method for minimizing multidimensional functions. *Chemical physics letters* 219, 5-6 (1994), 343–348.
- [25] Philippe Flajolet, Daniele Gardy, and Loÿs Thimonier. 1992. Birthday paradox, coupon collectors, caching algorithms and self-organizing search. *Discrete Applied Mathematics* 39, 3 (1992), 207–229.
- [26] Bacon D Gidney C and contributors. 2018. Cirq: A python framework for creating, editing, and invoking noisy intermediate scale quantum (NISQ) circuits. <https://github.com/quantumlib/Cirq>.
- [27] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. 2008. Quantum random access memory. *Physical review letters* 100, 16 (2008), 160501.
- [28] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 212–219.
- [29] Lov K Grover and Jaikumar Radhakrishnan. 2005. Is partial quantum search of a database any easier?. In *Proceedings of the seventeenth annual ACM symposium on Parallelism in algorithms and architectures*. 186–194.
- [30] Jacques Hadamard. 1893. Resolution d'une question relative aux determinants. *Bull. des sciences math.* 2 (1893), 240–246.
- [31] Aram W Harrow, Avinandan Hassidim, and Seth Lloyd. 2009. Quantum algorithm for linear systems of equations. *Physical review letters* 103, 15 (2009), 150502.
- [32] Akinori Hosoyamada and Yu Sasaki. 2018. Quantum Demirc-Selçuk meet-in-the-middle attacks: applications to 6-round generic Feistel constructions. In *International Conference on Security and Cryptography for Networks*. Springer, 386–403.
- [33] Nabil Itezhaz, M Kaykobad, and M Sohel Rahman. 2021. Multidimensional segment trees can do range updates in poly-logarithmic time. *Theoretical Computer Science* 854 (2021), 30–43.
- [34] Christian S Jensen, Dan Lin, and Beng Chin Ooi. 2004. Query and update efficient B⁺-tree based indexing of moving objects. In *Proceedings of the Thirtieth international conference on Very large data bases-Volume 30*. 768–779.
- [35] Ruslan Kapralov, Kamil Khadiev, Joshua Mokut, Yixin Shen, and Maxim Yagafarov. 2020. Fast Classical and Quantum Algorithms for Online k -server Problem on Trees. *arXiv preprint arXiv:2008.00270* (2020).
- [36] Iordanis Kerenidis and Anupam Prakash. 2017. Quantum Recommendation Systems. In *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- [37] Maria Kieferova, Ortiz Marrero Carlos, and Nathan Wiebe. 2021. Quantum Generative Training Using R⁺-enyi Divergences. *arXiv preprint arXiv:2106.09567* (2021).
- [38] Donald E Knuth. 1998. *The Art of Computer Programming: Sorting and Searching, volume 3*. Addison-Wesley Professional.
- [39] Jure Leskovec and Rok Sosič. 2016. Snap: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology (TIST)* 8, 1 (2016), 1–20.
- [40] Guangxi Li, Zhixin Song, and Xin Wang. 2021. VSQ: variational shadow quantum learning for classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 8357–8365.
- [41] Tongyang Li, Shouvanik Chakrabarti, and Xiaodi Wu. 2019. Sublinear quantum algorithms for training linear and kernel-based classifiers. In *International Conference on Machine Learning*. PMLR, 3815–3824.
- [42] Tongyang Li, Chunhao Wang, Shouvanik Chakrabarti, and Xiaodi Wu. 2021. Sublinear Classical and Quantum Algorithms for General Matrix Games. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 8465–8473.
- [43] Ashley Montanaro. 2015. Quantum speedup of Monte Carlo methods. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 471, 2181 (2015), 20150301.
- [44] Ashley Montanaro. 2017. Quantum pattern matching fast on average. *Algorithmica* 77, 1 (2017), 16–39.
- [45] Maria Naya-Plasencia and André Schrottenloher. 2020. Optimal Merging in Quantum k -xor and k -sum Algorithms. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 311–340.
- [46] Michael A Nielsen and Isaac L Chuang. 2001. Quantum computation and quantum information. *Phys. Today* 54, 2 (2001), 60.
- [47] Patrick O'Neil and Dallen Quass. 1997. Improved query performance with variant indexes. In *Proceedings of the 1997 ACM SIGMOD international conference on Management of data*. 38–49.
- [48] Mark H Overmars. 1983. *The design of dynamic data structures*. Vol. 156. Springer Science & Business Media.
- [49] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Informatica* 33, 4 (1996), 351–385.
- [50] Alexandru Paler, Oumarou Oumarou, and Robert Basmadjian. 2020. Parallelizing the queries in a bucket-brigade quantum random access memory. *Physical Review A* 102, 3 (Sept. 2020). <https://doi.org/10.1103/physreva.102.032608>
- [51] Patrick Rebertost, Masoud Mohseni, and Seth Lloyd. 2014. Quantum support vector machine for big data classification. *Physical review letters* 113, 13 (2014), 130503.
- [52] Yue Ruan, Xiling Xue, Heng Liu, Jianing Tan, and Xi Li. 2017. Quantum algorithm for k -nearest neighbors classification based on the metric of hamming distance. *International Journal of Theoretical Physics* 56, 11 (2017), 3496–3507.
- [53] Seyran Saeedi and Tom Arodz. 2019. Quantum sparse support vector machines. *arXiv preprint arXiv:1902.01879* (2019).
- [54] Manuel Schöninger. 2022. Applicability of quantum computing on database query optimization. In *Proceedings of the 2022 International Conference on Management of Data*. 2512–2514.
- [55] Erwin Schrödinger. 1935. Discussion of probability relations between separated systems. In *Mathematical Proceedings of the Cambridge Philosophical Society*, Vol. 31. Cambridge University Press, 555–563.

- [56] Pranab Sen and Srinivasan Venkatesh. 2001. Lower bounds in the quantum cell probe model. In *International Colloquium on Automata, Languages, and Programming*. Springer, 358–369.
- [57] Peter W Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 124–134.
- [58] Peter W Shor. 1998. Quantum computing. *Documenta Mathematica* 1, 1000 (1998), 467–486.
- [59] Andrew Steane. 1998. Quantum computing. *Reports on Progress in Physics* 61, 2 (1998), 117.
- [60] Immanuel Trummer and Christoph Koch. 2015. Multiple query optimization on the D-Wave 2X adiabatic quantum computer. *arXiv preprint arXiv:1510.06437* (2015).
- [61] Valter Uotila. 2022. Synergy between Quantum Computers and Databases. (2022).
- [62] Robert Wille, Rod Van Meter, and Yehuda Naveh. 2019. IBM’s Qiskit tool chain: Working with and developing for real quantum computers. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1234–1240.
- [63] Yanbing Xue and Milos Hauskrecht. 2017. Efficient learning of classification models from soft-label information by binning and ranking. In *The Thirtieth International Flairs Conference*.
- [64] Baoyue Yan, Xuntao Cheng, Bo Jiang, Shibin Chen, Canfang Shang, Jianying Wang, Gui Huang, Xinjun Yang, Wei Cao, and Feifei Li. 2021. Revisiting the design of LSM-tree based OLTP storage engine with persistent memory. *Proceedings of the VLDB Endowment* 14, 10 (2021), 1872–1885.
- [65] Andrew Chi-Chih Yao. 1981. Should tables be sorted? *Journal of the ACM (JACM)* 28, 3 (1981), 615–628.
- [66] Rui Zhang, Jianzhong Qi, Dan Lin, Wei Wang, and Raymond Chi-Wing Wong. 2012. A highly optimized algorithm for continuous intersection join queries over moving objects. *The VLDB journal* 21 (2012), 561–586.

A DETAILS OF DYNAMIC VARIANT

In this section, we introduce how to make the static quantum B+ tree dynamic, which solves the dynamic range query. One idea of designing our quantum B+ tree is to retain a classical B+ tree structure and to maintain a concise replication of hierarchical relationships in QRAMs. This enables the flexible extension to the B+ tree variants that have been well studied in classical computers. As such, we adapt the idea of the logarithmic method [10] to enable the insertion operations of the quantum B+ tree by building some forests on the quantum B+ tree. Based on the forests, we propose our approach to perform deletions. In the following, the details of insertion and deletion are discussed in Section A.1, and then in Section A.2, we introduce how to solve the dynamic quantum range query with the dynamic quantum B+ tree.

A.1 Insertion and Deletion

Following the idea of the logarithmic method [10], we build at most L forests, says $F_0, \dots, F_{L-1}$, where $L = \lfloor \log_B N \rfloor + 1$, and for each $i \in [0, L - 1]$, the forest F_i contains at most $B - 1$ static quantum B+ trees of height i .

To insert a new key-record pair (*key*, *rec*), we insert it into a sorted list first. When the length of this sorted list reaches B , we flush it into F_0 . Then, whenever a forest F_i has B quantum B+ trees, which indicates that we have B quantum B+ trees of height i , we merge the B quantum B+ trees of height i into a quantum B+ trees of height $i + 1$, and add it into F_{i+1} .

In additional to the classical B+ tree, we also replicate the insertion in the quantum B+ tree such that the hierarchical relationships in the QRAM are consistent with the classical component.

Next, we discuss how to delete a key-record pair in a dynamic quantum B+ tree. It involves two steps. The first step is to locate the quantum B+ tree in the forests which contains the key-record pair. The second step is to delete the key-record pair in both the

classical component and the quantum component of the quantum B+ tree.

To locate the tree containing a key-record pair, we assign each key-record pair a unique ID in ascending order (e.g., by a counter). We also maintain an auxiliary B+ tree $\mathcal{T}_0$ to store the one-to-one mapping from the key-record pair to its ID. When a key-record pair is inserted, we insert the key-record pair and its ID into $\mathcal{T}_0$. When a key-record pair is deleted, we delete it in $\mathcal{T}_0$ accordingly. Furthermore, we maintain another auxiliary B+ tree $\mathcal{T}_1$ to store the mapping from the key-record pair ID to the forest F_i it belongs to. Similar to $\mathcal{T}_0$, we do insertions and deletions in $\mathcal{T}_1$ accordingly. In addition, when we merge F_i , we update $\mathcal{T}_1$ for all the key-record pairs in F_i .

To delete a key-record pair (*key*, *rec*) in a B+ tree in F_i , we do a classical search to find the leaf that contains (*key*, *rec*) and replace the key-record pair with a *dummy*. Then, we check if its ancestors are still balanced. If an imbalanced ancestor is found, we first check if it can borrow a child from its sibling. Figure ?? shows an example in this case. Otherwise, we check if it can be directly merged with its sibling, which indicates that the node and its sibling have at most B children. If not, we merge the node and its sibling by rebuilding the subtrees below them. Figure ?? shows an example in this case. Then, after rebalancing the B+ tree, we check if the root node still has at least two children. If not, we check if it can borrow a child from another B+ tree in F_i . If not, then if there are at least two B+ trees in F_i , we merge the root nodes of the two B+ trees. Otherwise, we remove the root node and downgrade the B+ tree from F_i to F_{i-1} .

The following lemma shows that the insertions and deletions in the quantum B+ tree is efficient.

LEMMA A.1. *The amortized cost of an insertion and a deletion in a dynamic quantum B+ tree is $O(C_s \log N)$ and $O((C_s + C_l) \log N)$, respectively, where C_s (C_l) is the cost of a store (load) operation on QRAM.*

PROOF SKETCH. By Theorem 3.1 in [10], N insertions totally cost $O(N \log N)$ time. Moreover, each insertion involves store operations on QRAM to at most $L (= \lfloor \log_B N \rfloor + 1)$ forests. Therefore, the amortized cost of one insertion is $O(C_s \log N)$. For deletions, the first step costs $O(C_l \log N)$ time, since it consists of two point queries in B+ trees. By the analysis of partial rebuilding in [48], the second step also costs $O(C_s \log N)$ time (which takes into consideration the store operations on at most L forests similar to insertions). Therefore, the amortized cost of a deletion is $O((C_s + C_l) \log N)$. $\square$

Still, both C_s and C_l can be skipped as they are considered to be constants by [32, 35, 37, 41, 42, 44, 45]. Thus, the amortized cost of an insertion and a deletion in a dynamic quantum B+ tree can both be written as $O(\log N)$ for simplicity.

Moreover, the total space cost of the dynamic quantum B+ tree is $O(BN \log_B N)$, as we maintain $L = \lfloor \log_B N \rfloor + 1$ forests, each containing at most $B - 1$ static quantum B+ trees, and each quantum B+ tree takes $O(N)$ space (based on Theorem 4.5). Since B is a commonly regarded as a constant, the space cost of the dynamic quantum B+ tree can be simplified as $O(N \log N)$.

A.2 Query

To answer a query $QUERY(x, y)$ on our dynamic quantum B+ tree, we extend our idea of the GCLQ algorithm to perform both a global classical search and a local quantum search on the forests $F_0, \dots, F_{L-1}$.

First, for global classical search, we follow the similar global search steps as introduced in Section 4.2 for each tree in the forests. Specifically, we first create an empty list $\mathcal{L}_i$ for each forest F_i ($i \in [0, L-1]$), and we add all the root nodes in F_i into $\mathcal{L}_i$. Then, for each list $\mathcal{L}_i$, we perform a global classical search from each root in $\mathcal{L}_i$. A candidate set of at most two nodes are returned for each root, which are then added to the original list for easier processing.

Then, we perform the local quantum search for the candidate nodes in the lists $\mathcal{L}_0, \dots, \mathcal{L}_{L-1}$. Consider the nodes $u_0, u_1, \dots, u_{m-1}$ in the lists, where m is the total number of nodes in $\mathcal{L}_0, \dots, \mathcal{L}_i$. We initialize the quantum bits to be $\sum_{i=0}^{m-1} \frac{\sqrt{B^{h(u_i)+1}}}{\sqrt{\sum_{j=0}^{m-1} B^{h(u_j)+1}}} |u_i\rangle$, where

$h(u_i)$ is the height of u_i , and $\frac{\sqrt{B^{h(u_i)+1}}}{\sqrt{\sum_{j=0}^{m-1} B^{h(u_j)+1}}}$ is the normalized am-

plitude of each u_i such that each of the key-record pair below the nodes in the lists has the same amplitude in the result. Then, we do the same steps in Section 4.2.2 to obtain the leaves below the nodes. Finally, we do a post-selection search as discussed in Section 4.2.

The following theorem shows the time complexity for the dynamic quantum B+ tree to answer a quantum range query.

THEOREM A.2. *On average, the dynamic quantum B+ tree answers a range query in $O(C_l \log^2 N)$ time.*

PROOF SKETCH. The global classical search costs $O(C_l \log^2 N)$ time, since we have $O(\log N)$ forests and the global search for each forest needs $O(C_l \log N)$ time. For the local quantum search, the initialization and the steps to obtain the leaves cost $O(C_l \log N)$. The average number of post-selection is $O(\log N)$, since there are at most $2B \lceil \log_B N \rceil$ nodes in the lists. Totally, the time complexity is $O(C_l \log^2 N)$. $\square$

Similarly, the time complexity $O(C_l \log^2 N)$ can be simplified as $O(\log^2 N)$ by [32, 35, 37, 41, 42, 44, 45].

B COMPLETE PROOFS

In this section, we show the complete proofs of all the lemmas and theorems in this paper.

PROOF OF LEMMA 4.1. Consider a static quantum range query $QUERY(x, x)$. It answers whether the key x exists in the dataset, which is a static membership problem. By [56], the time complexity of answering a static membership problem in quantum computers with the QRAM model is formed by $\Omega(\log N)$ QRAM load operations, each costing C_l . Therefore, the time complexity of answering a static quantum range query is $\Omega(C_l \log N)$. $\square$

PROOF OF LEMMA 4.2. Let N be the number of non-dummy key-record pairs stored in the weight-balanced B+ tree, and let $B \geq 4$ be the branching factor. Let M be the total number of nodes (internal nodes and leaves), and let H be the height of the tree (i.e., the height of the root). Recall that the height of a leaf is 0, that all leaves lie at the same level, and that a non-root node of

height h is balanced if and only if its weight lies in $[\frac{1}{4}B^{h+1}, B^{h+1}]$. By definition of a weight-balanced B+ tree, every non-root node is balanced and the root has at least two non-dummy children; in particular, the root is not a leaf, so every leaf is a non-root node of height 0. We first bound the number of nodes at each height. For every $h \in \{0, 1, \dots, H\}$, every key-record pair belongs to exactly one node of height h (namely, the unique node of height h on the root-to-leaf path leading to that pair). Hence the nodes of height h form a partition of the N key-record pairs. Write n_h for the number of nodes of height h . Then $n_H = 1$ (the root). For every $h \in \{0, 1, \dots, H-1\}$, every node of height h is a non-root node and is therefore balanced, so its weight is at least $\frac{1}{4}B^{h+1}$. Since these n_h nodes partition N key-record pairs,

$$n_h \leq \frac{N}{\frac{1}{4}B^{h+1}} = \frac{4N}{B^{h+1}}.$$

In particular, taking $h = 0$, every leaf contains at least $\frac{B}{4}$ (and at most B) non-dummy key-record pairs, and the number of leaves satisfies

$$n_0 \leq \frac{4N}{B} = O\left(\frac{N}{B}\right).$$

Summing over all heights,

$$\begin{aligned} M &= \sum_{h=0}^H n_h \leq 1 + \sum_{h=0}^{H-1} \frac{4N}{B^{h+1}} = 1 + \frac{4N}{B} \sum_{h=0}^{H-1} B^{-h} \\ &\leq 1 + \frac{4N}{B} \sum_{h=0}^{\infty} B^{-h} = 1 + \frac{4N}{B} \cdot \frac{1}{1 - \frac{1}{B}} = 1 + \frac{4N}{B-1}. \end{aligned}$$

Thus $M = O(N/B)$. Equivalently, the number of non-leaf nodes is

$$M - n_0 = 1 + \sum_{h=1}^{H-1} n_h \leq 1 + \sum_{h=1}^{H-1} \frac{4N}{B^{h+1}} = O\left(\frac{N}{B}\right)$$

as well (cf. [38]). By construction, the hierarchy QRAM $\mathcal{Q}_0$ and the data QRAM $\mathcal{Q}_1$ each store $M \cdot B$ entries (one entry per child slot of each node). Therefore the total number of memory blocks occupied by the quantum B+ tree is

$$2 \cdot M \cdot B = O\left(\frac{N}{B}\right) \cdot B = O(N),$$

as claimed. $\square$

PROOF OF LEMMA 4.3. Since the routing keys of all nodes in the same level are disjoint, we cannot have more than two partial nodes in the same level. It is also easy to verify that the returned nodes are from the same level. Thus, the candidate set contains at most two nodes. Since we only filter out the outside node in this algorithm, we have $\mathcal{R}^* \subset \mathcal{R}$. Finally, since the precise partial node either is a leaf (which contains at least $1/B$ items in the query range), or contains an inside child (which contains at least $\frac{1}{4B}$ items in the query range due to the balanced nodes of the B+ tree), and it can be verified that at least one returned node is precise, we have $|\mathcal{R}^*| \geq \frac{1}{8B} |\mathcal{R}|$. $\square$

PROOF OF THEOREM 4.4. We analyze the time cost of the global classical search and the local quantum search separately.

Global classical search. At each tree level, the global classical search returns at most two candidate nodes. Since the height of a weight-balanced quantum B+ tree with N key-record pairs is $O(\log N)$, the search traverses at most $O(\log N)$ levels. At each level, the algorithm performs a constant number of QRAM load

operations, and each load operation costs C_l time. Therefore, the global classical search performs $O(\log N)$ QRAM loads and takes $O(C_l \log N)$ time on average.

Local quantum search. Suppose the global classical search stops at two candidate nodes u and v at tree level j , and let h be the height of the root. By the trigger condition for local quantum search, all non-dummy key-record pairs below one child of u and one child of v belong to the query answer. Let k be the number of such key-record pairs. By the weight-balance property of the quantum B+ tree, we have $k \geq \frac{1}{4}B^{h-j}$. On average, the local quantum search repeats all its steps for $\frac{2B^{h-j+1}}{k} \leq 8B$ times. Since B is a constant parameter, this is $O(1)$ repetitions on average. By Lemma 4.3, the number of repetitions is at most $8B$ in the worst case, which is also a constant. In each repetition, Steps 2 and 3 are executed at most $O(\log N)$ times. Each execution of Step 2 or Step 3 runs a quantum circuit for post-selection, whose cost is dominated by a constant number of QRAM load operations, each costing C_l time. Hence, one repetition of the local quantum search costs $O(C_l \log N)$ time on average, and the entire local quantum search costs $O(C_l \log N)$ time on average.

Total time. Combining the two parts, the quantum range query algorithm returns the answer in $O(C_l \log N)$ time on average. $\square$

PROOF OF THEOREM 4.5. We analyze the space cost in three parts: the QRAM storage of the quantum B+ tree, the classical B+ tree structure, and the extra qubits used in local quantum search.

First, by Lemma 4.2, the quantum B+ tree is stored in two QRAMs, Q_0 and Q_1 , using $O(N)$ memory blocks in total. Since each memory block stores a constant amount of information, the QRAM space cost is $O(N)$.

Second, the underlying classical weight-balanced B+ tree also occupies $O(N)$ space. Each non-dummy key-record pair is stored once, and the number of nodes is $O(N/B) = O(N)$; each node stores $O(B)$ routing keys or key-record pairs, so the total classical storage is $O(N)$ [38]. Thus, maintaining the classical B+ tree does not increase the asymptotic space bound beyond $O(N)$.

Third, during local quantum search, the algorithm allocates temporary quantum registers beyond the QRAM. Specifically, it uses $2n_i + n_b$ additional qubits, where n_i and n_b are constants determined by the node index size and the branching factor B , respectively. Since B is a fixed user parameter and $n_i = O(\log N)$ qubits suffice to address $O(N)$ memory blocks, the extra qubit count $2n_i + n_b$ is $O(\log N)$ in the worst case. However, these registers are reused across iterations and are not required to persist after the query completes, so they contribute only a constant-factor overhead relative to the $O(N)$ QRAM storage when measuring overall space cost.

Combining the above, the total space cost of the quantum B+ tree is $O(N)$. $\square$

PROOF OF LEMMA A.1. By Theorem 4.5 in this paper and Theorem 3.1 in [10], N insertions totally cost $O(N \log N)$ time. Therefore, the amortized cost of one insertion is $O(\log N)$. Moreover, each insertion involves store operations on QRAM for at most $L = \lceil \log_B N \rceil + 1 = O(\log N)$ forests, and each store operation costs C_s time. Thus, the amortized cost of one insertion is $O(C_s \log N)$.

To analyze the update cost in a merge, we find that when merging F_i , the time complexity to update $\mathcal{T}_1$ is $O(\log N)$. The reason is as follows.

Let ID_l denote the smallest ID in F_i . For each $j < i$ and each key-record pair in F_j , the ID of the key-record pair is smaller than ID_l . Let ID_r denote the largest ID in F_i . Then, for each $j > i$ and each key-record pair in F_j , the ID of the key-record pair is greater than ID_r . Therefore, the update operations for the key-record pairs in F_i can be merged into a range update. Then, we can use lazy propagation [33] to do the range update in $O(\log N)$ time.

Thus, the extra cost to maintain $\mathcal{T}_0$ and $\mathcal{T}_1$ has no impact, which means the amortized cost of insertion is still $O(C_s \log N)$.

For deletions, the first step costs $O(C_l \log N)$ time, since it consists of two point queries in B+ trees.

Then consider the second step. Motivated by the analysis of partial rebuilding in [48], we consider a node u of height h just after a rebuild. The node u is perfectly balanced such that its weight $w(u) \geq \frac{1}{2}B^{h+1}$. Since it will become imbalanced if and only if $w(u) < \frac{1}{4}B^{h+1}$, there must be $\Omega(B^{h+1})$ deletions below the node u or its siblings before that. So, it is charged $O(1)$ time for each deletion below it and its siblings. Then, for a deletion in a leaf node, each ancestor and its siblings are charged $O(1)$ time, so they are totally charged $O(\log N)$ time. Moreover, similar to insertions, each deletion involves store operations on QRAM for at most $L = O(\log N)$ forests, and each store operation costs C_s time. Therefore, the amortized cost of a deletion is $O((C_s + C_l) \log N)$. $\square$

PROOF OF THEOREM A.2. The global classical search has a cost of $O(C_l \log^2 N)$ time, since there are $O(\log N)$ forests and the global search for each forest needs $O(C_l \log N)$ time. Consider the cost of the local quantum search. The initialization and the steps to obtain the leaves cost $O(C_l \log N)$. Then, we multiply it by the average post-selection times. Let N' denote the total number of key-record pairs below the nodes in $\mathcal{L}_0, \dots, \mathcal{L}_{L-1}$. Then, the post-selection needs to be iterated $O(N'/k)$ times on average. Since there are at most $2B \lceil \log_B N \rceil$ nodes in the lists and each node has a weight at most B^{i+1} , we have $N' \leq 2B^{i+2} \log_B N$. Before we turn to the local quantum search, we have found a child of a node in $\mathcal{L}_i$ such that all the key-record pairs below the child are in the answer, such that $k \geq \frac{1}{4}B^i$. Therefore, $N'/k = O(\log N)$. Hence, the local quantum search costs $O(C_l \log^2 N)$ time on average.

Since both the global classical search and the local quantum search cost $O(C_l \log^2 N)$ time, the dynamic quantum B+ tree answers a range query in $O(C_l \log^2 N)$ time on average. $\square$

C REMAINING EXPERIMENTAL RESULTS

In this section, we present the remaining experimental results. First, Figure 5(a) shows that, as the deletion–insertion ratio varies from 1% to 5%, the IO cost and execution time of both the classical and quantum B+ trees remain nearly unchanged, indicating that our **quantum B+ tree** delivers stable performance under insertion and deletion. Second, Figures 5(b) and (c) show that, for search queries over *IMDb*, the **quantum B+ tree** attains similarly strong gains over the baselines as for maximum queries. Third, we applied our **quantum B+ tree** in join algorithm selection of query optimization,

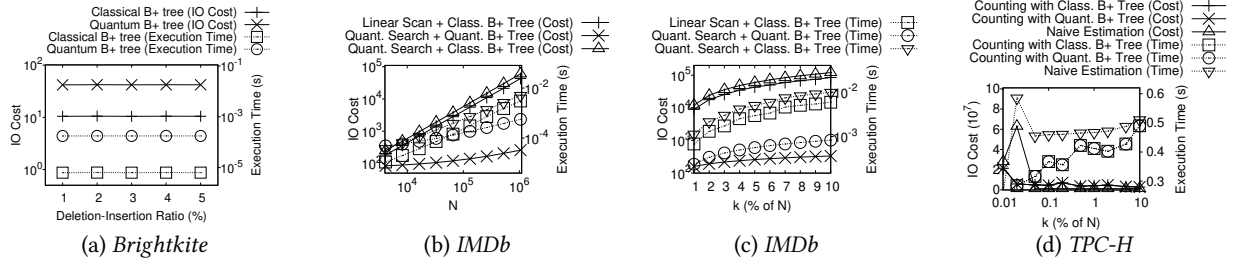


Figure 5: (a) The Effect of Deletion-Insertion Ratio of the Dynamic Data Structures on Dataset *Brightkite*, the Effect of (b) N and (c) k of Search Query on Dataset *IMDb*, and (d) Query Optimization Results for Join Algorithm Selection on Dataset *TPC-H*

which is similar to that in join order selection. Specifically, we ran a query where Table *Orders* (with 300,000 records) is joined with Table *Lineitems* (with 1,200,000 records) with the where-clause of attribute *totalprice* in Table *Orders* is within a random range. In this query, when the number of feasible records in Table *Orders* satisfying the where-clause is sufficiently small (i.e., ≤ 50 according to our experimental results), we select nested loop join algorithm to

achieve better query performance, and otherwise, hash join will be selected. This is because, when one table to be joined has very small number of records, the hash join algorithm needs more overhead time for building the hash, leading to overall slower query time. As shown in Figure 5(d), join algorithm selection yields similar trends: the **quantum B+ tree** again outperforms classical counting and proportional estimation in both IO cost and execution time.